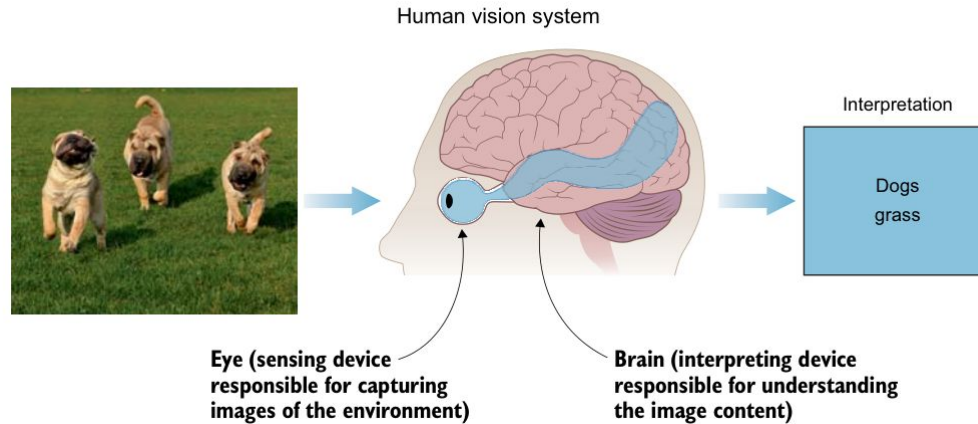


Unidad 9

Arquitectura CNN: Capas de Convolución, filtros/kernels y mapas de características (*feature maps*). Operaciones Espaciales: Padding, Stride y Capas de Pooling (Max Pooling, Average Pooling). Capas Densas Finales: Transición de patrones espaciales a clasificación/regresión. Transfer Learning: Uso de modelos pre-entrenados (ResNet, VGG, Inception) y Fine-tuning.

Computer Vision



La **percepción visual**, en su forma más básica, es el **acto de observar patrones y objetos a través de la vista o la información visual**.

En su nivel más básico, los sistemas de visión son prácticamente iguales para humanos, animales, insectos y la mayoría de los organismos vivos. Constan de un sensor o un ojo para capturar la imagen y un cerebro para procesarla e interpretarla. El sistema genera entonces una predicción de los componentes de la imagen basándose en los datos extraídos de la misma.

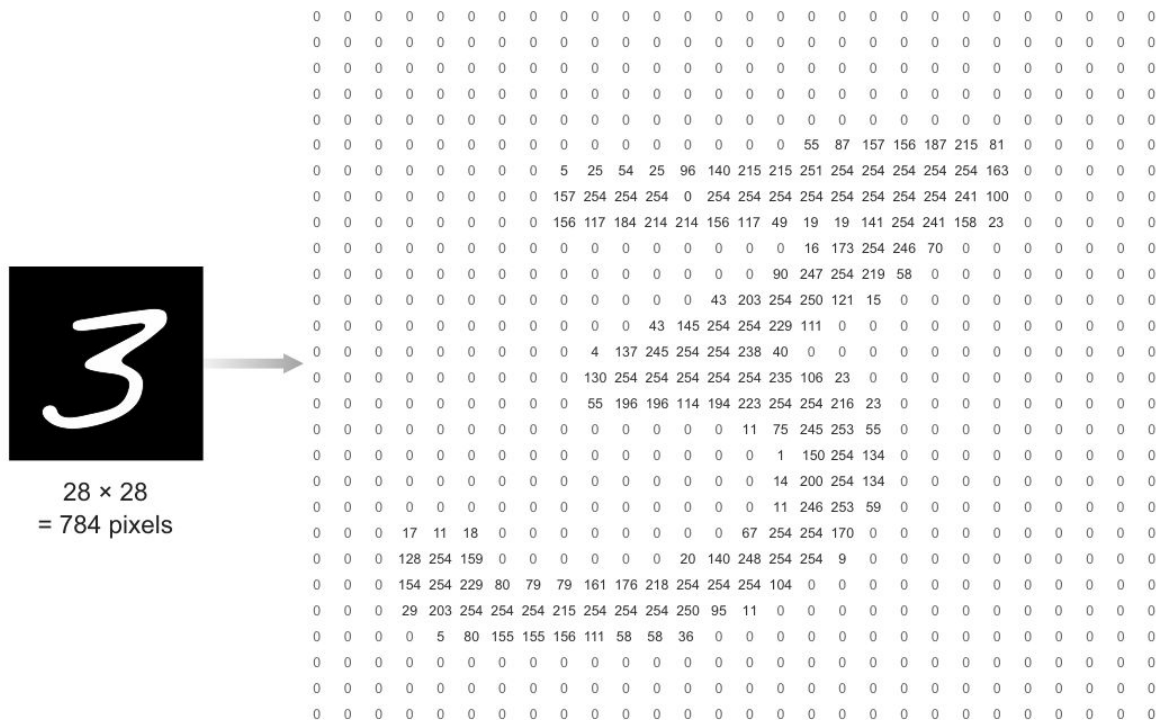


Figure 3.2 The computer sees this image as a 28×28 matrix of pixel values ranging from 0 to 255.

Cómo ve una computadora

What we see

What computers see



08	02	22	97	38	15	00	40	00	75	04	05	07	78	52	12	50	77	91	08
49	49	99	40	17	81	18	57	60	87	17	40	98	43	69	46	04	56	62	00
81	49	31	73	55	79	14	29	93	71	40	67	53	99	30	03	49	13	36	65
52	90	95	23	04	60	11	42	69	24	48	56	01	32	54	71	37	02	34	91
22	31	14	71	51	67	43	59	41	92	34	54	22	40	40	28	44	33	13	80
24	47	32	60	99	03	45	02	44	75	33	53	78	36	64	20	35	09	12	80
32	98	81	28	64	23	67	10	26	38	40	67	59	54	70	66	18	38	64	70
47	24	20	68	02	62	12	20	95	63	94	39	63	04	49	91	44	49	94	21
24	55	58	05	66	73	99	26	97	17	78	78	94	83	14	88	34	89	63	72
21	36	23	09	75	00	74	44	20	45	35	14	00	41	33	97	34	31	33	95
78	17	53	28	22	75	31	67	15	94	03	80	04	42	16	14	09	53	56	92
16	39	05	42	96	35	31	47	55	58	88	24	00	17	54	24	34	29	85	57
84	56	00	48	35	71	89	07	05	44	44	37	44	40	21	58	51	54	17	58
19	80	61	68	05	94	47	49	28	73	92	13	86	52	17	77	04	09	55	40
04	52	08	83	97	35	99	14	07	97	57	32	16	26	26	79	33	27	98	44
04	36	00	01	57	62	20	72	03	10	33	07	40	55	12	32	43	93	53	09
04	42	14	73	38	25	39	11	24	94	72	18	06	46	29	32	40	62	74	36
20	49	34	41	72	30	23	88	34	62	99	69	82	47	59	85	74	04	34	24
20	23	35	29	78	31	90	01	74	31	49	71	48	86	81	14	23	57	05	54
01	70	54	71	83	51	54	49	16	92	33	48	61	43	51	01	89	19	67	4P

Figure 3.32 To a computer, an image looks like a 2D matrix of pixel values.

Canales RGB de una imagen

Color image



$F(0, 0) = [11, 102, 35]$

RGB channels

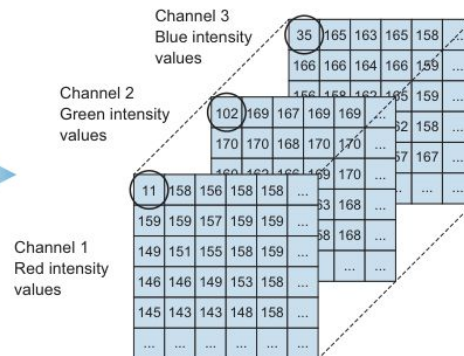


Figure 3.33 Color images are represented by three matrices. Each matrix represents the value of its color's intensity. Stacking them creates a complete color image.

Classification



a

CAT

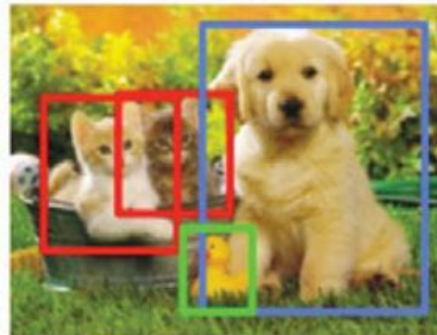
Classification
+ Localization



b

CAT

Object Detection



c

CAT, DOG, DUCK

Instance Segmentation



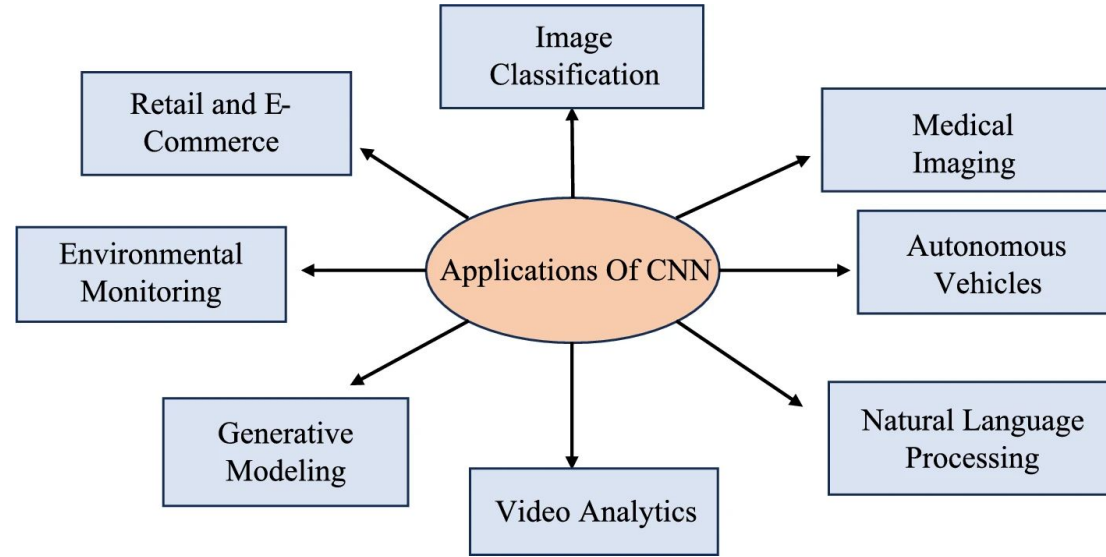
d

CAT, DOG, DUCK

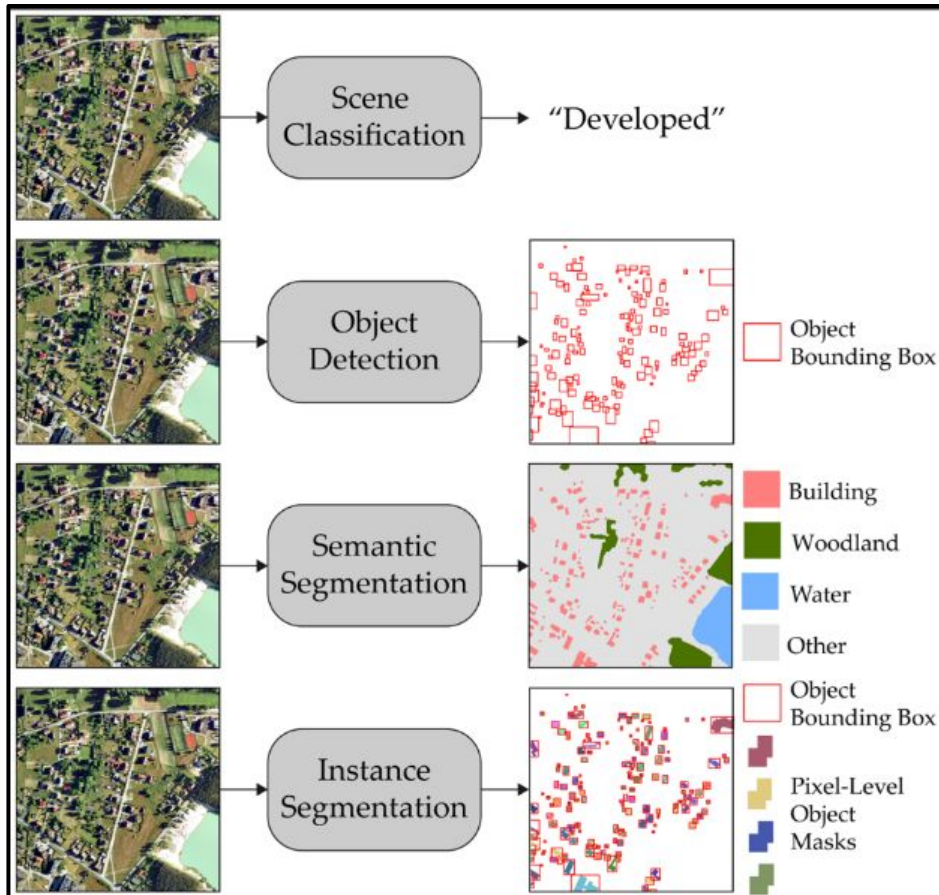
Dada una imagen como la que se muestra en la figura 1.1a, los humanos pueden ver fácilmente un "gato" en la imagen y, por lo tanto, categorizar la imagen (**tarea de clasificación**); localizar al gato en la imagen (**tarea de clasificación y localización**, como se muestra en la figura 1.1b); localizar y etiquetar todos los objetos presentes en la imagen (**tarea de detección de objetos**, como se muestra en la figura 1.1c); y segmentar los objetos individuales presentes en la imagen (**tarea de segmentación de instancias**, como se muestra en la figura 1.1d).

Principales tareas en computer vision

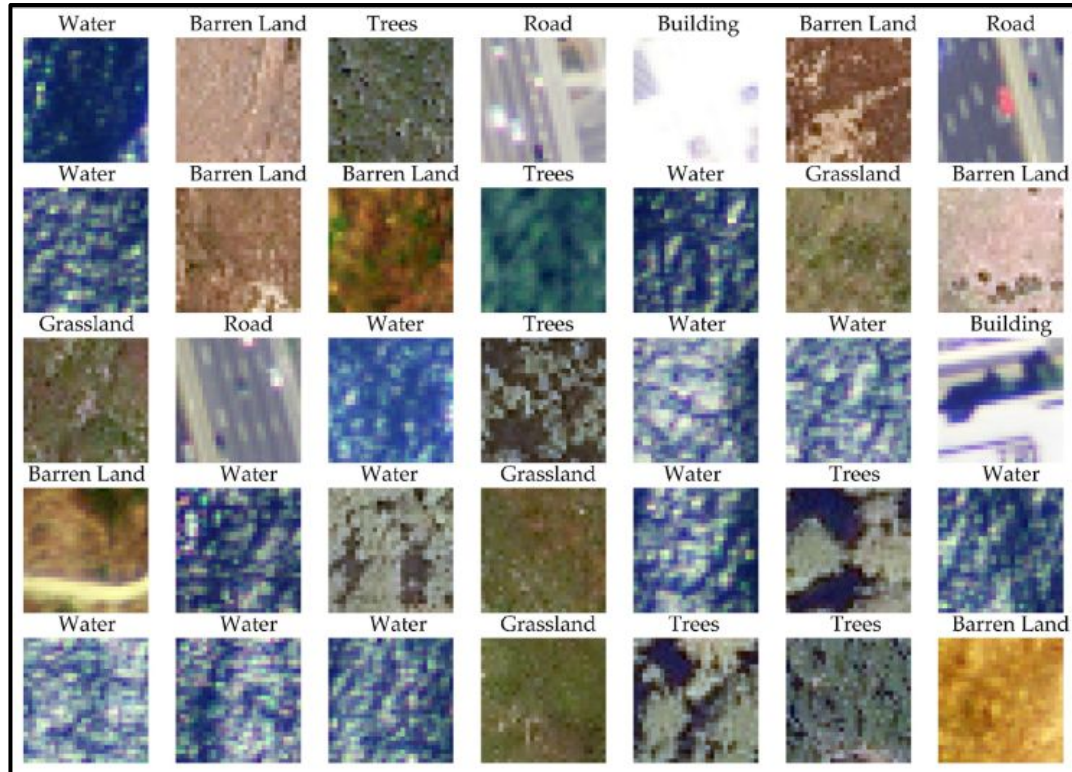
- Clasificación de imágenes
- Detección de objetos y localización
- Generar arte (style transfer)
- Creación de imágenes
- Reconocimiento facial
- Sistemas de recomendación de imágenes



Aplicaciones geoespaciales



Aplicaciones geoespaciales



Arquitectura de redes convolucionales

Las redes neuronales convolucionales (conocidas como CNN o ConvNets) son un **tipo especializado de red neuronal diseñado para procesar datos que tienen una topología conocida en forma de cuadrícula**, como las imágenes (una cuadrícula 2D de píxeles) o los datos de series temporales (una cuadrícula 1D). Matemáticamente, su principal característica es que **utilizan la operación lineal de convolución en lugar de la multiplicación general de matrices en al menos una de sus capas**.

A diferencia de los perceptrones multicapa tradicionales (donde cada neurona está conectada a todas las demás y la imagen debe aplanarse, perdiendo información espacial), las CNN **toman la matriz de la imagen cruda como entrada**, lo que ayuda enormemente a la red a comprender los patrones contenidos en los valores de los píxeles sin perder el contexto espacial.

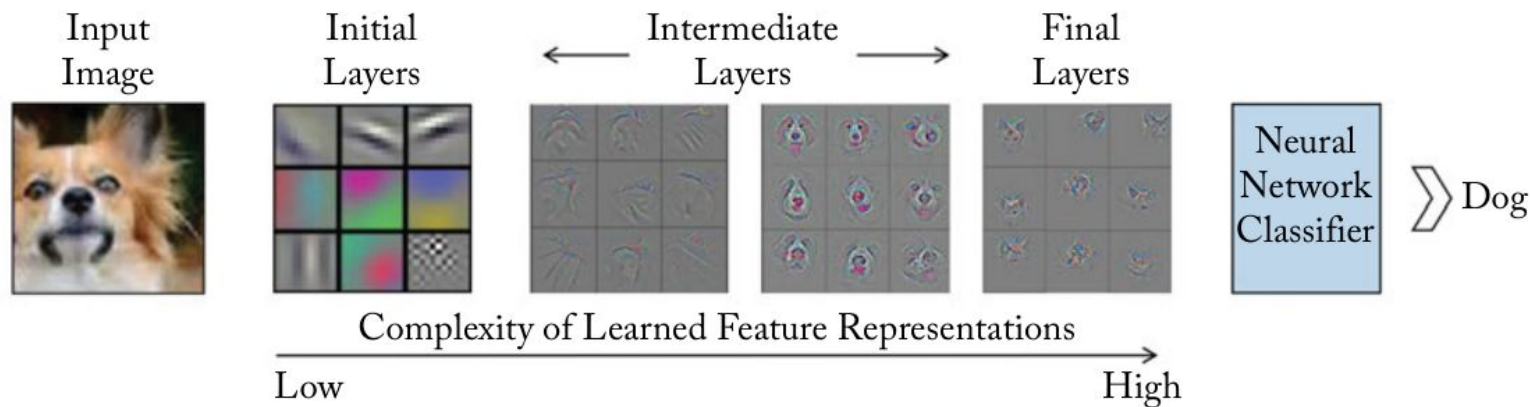
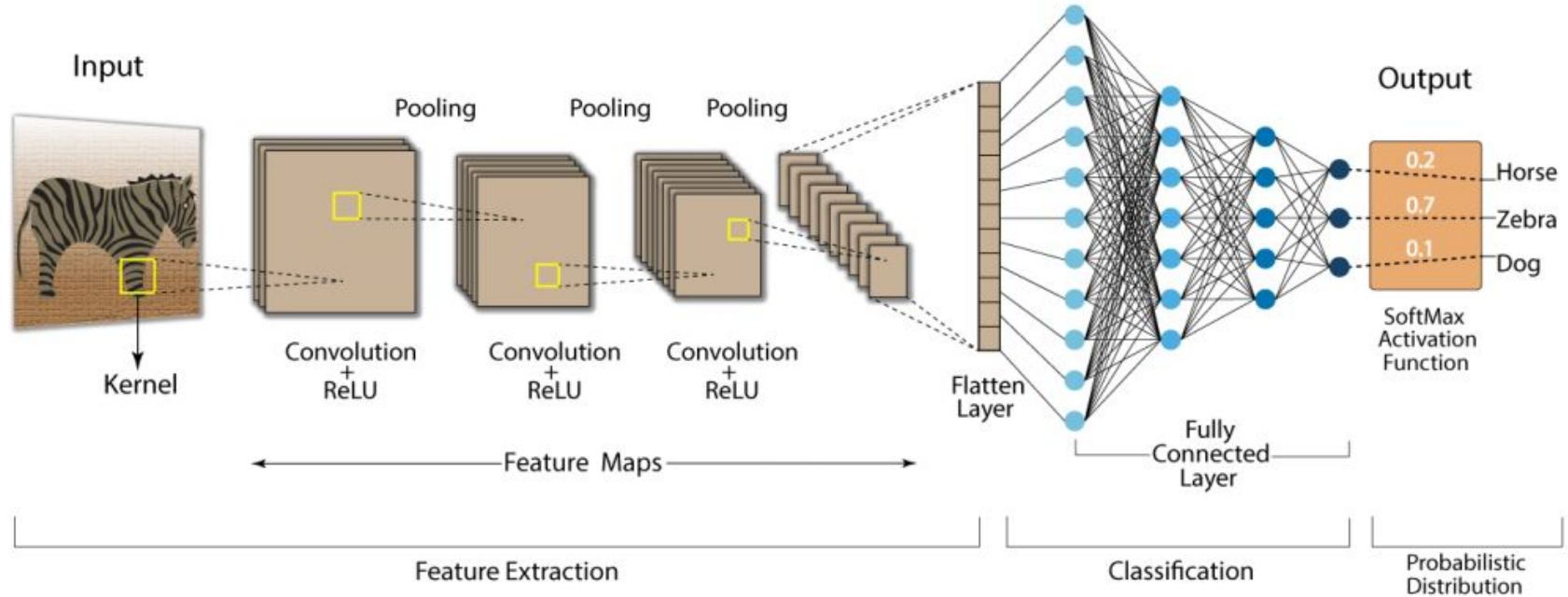


Figure 4.1: A CNN learns low-level features in the initial layers, followed by more complex intermediate and high-level feature representations which are used for a classification task. The feature visualizations are adopted from [Zeiler and Fergus \[2014\]](#).

Arquitectura de redes neuronales Convolucionales

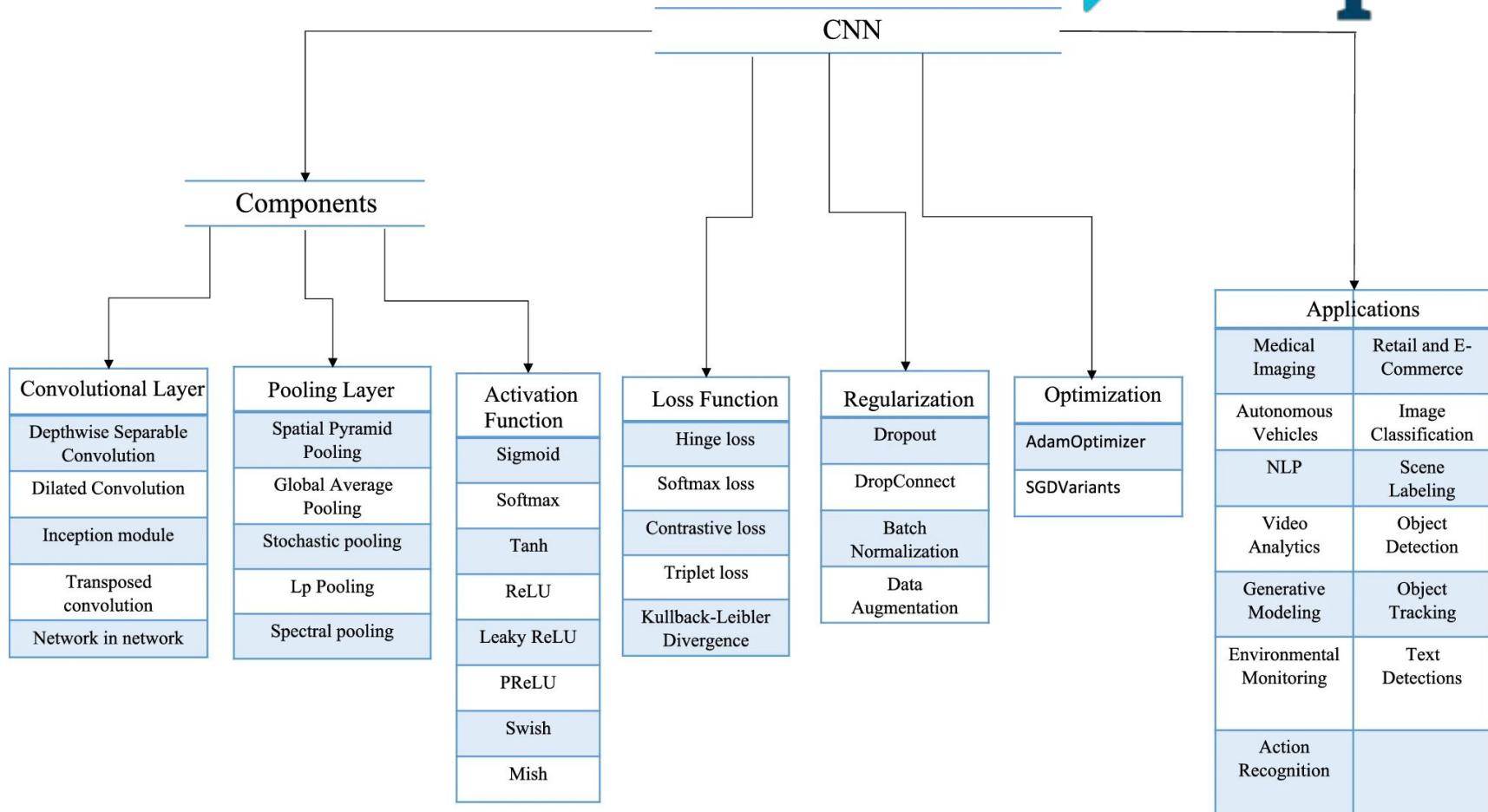


Además, las CNN **aprenden jerarquías espaciales de patrones**. Las primeras capas de la red aprenden a extraer características simples y de bajo nivel (como líneas y bordes locales), mientras que las capas más profundas combinan estas características para detectar conceptos visuales cada vez más abstractos y complejos (como texturas, formas, o partes específicas de un objeto como el hocico de un perro o la rueda de un auto).

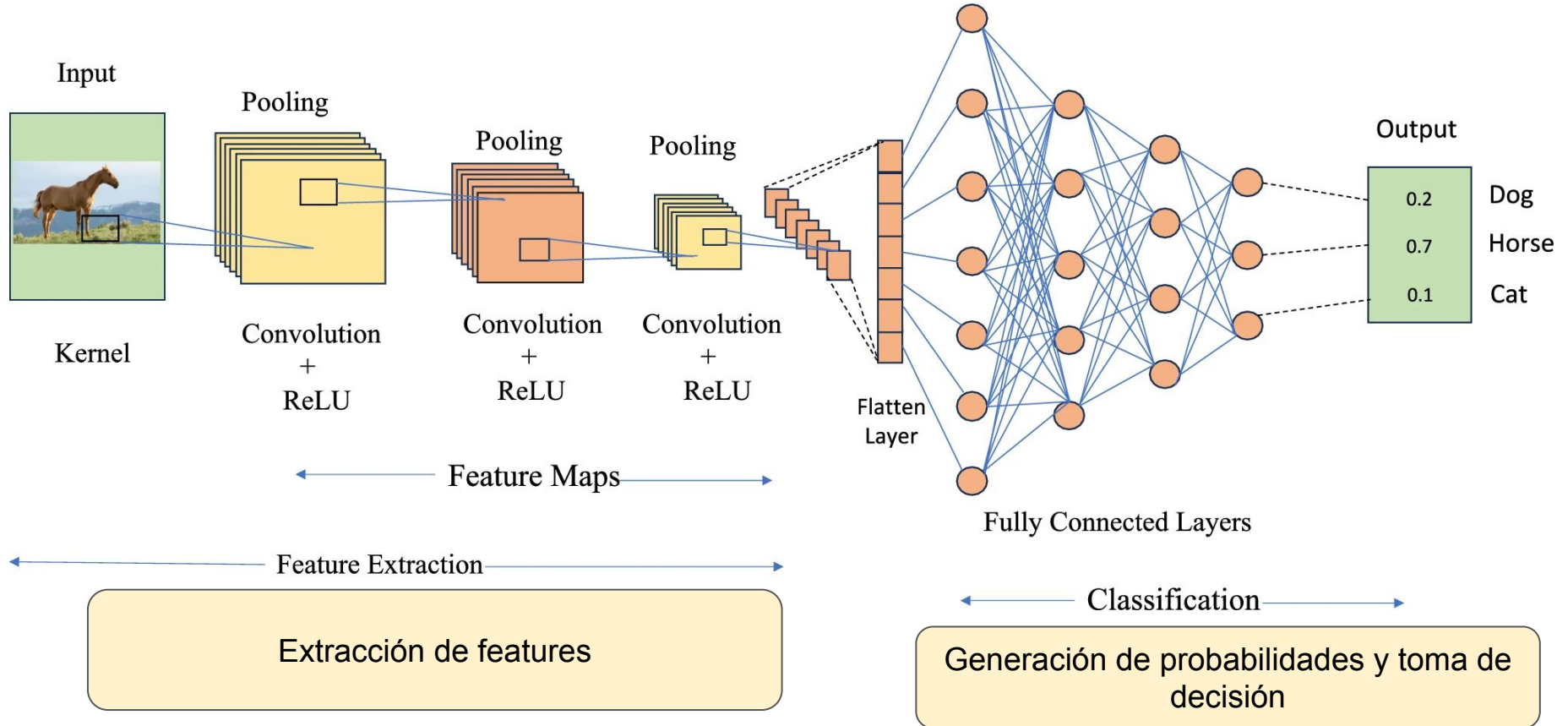
Para lograr esto, la arquitectura de una CNN suele estar compuesta por tres tipos básicos de capas:

1. **Capa Convolutiva:** Es el bloque central encargado de extraer características al deslizar los filtros sobre la imagen para crear un "mapa de características".
2. **Capa de Agrupación (Pooling):** Reduce la dimensionalidad (submuestreo) de los mapas de características, condensando la información y aportando mayor invarianza a pequeños cambios de escala o posición.
3. **Capas Totalmente Conectadas (Fully Connected):** Tras la extracción de características, los datos se aplanan y se pasan a una red neuronal tradicional para clasificar la imagen y emitir una predicción final.

Partes de una CNN



Etapas de procesamiento de la información



Convolución

$$(f * g)(t) \stackrel{\text{def}}{=} \int_{-\infty}^{\infty} f(\tau) g(t - \tau) d\tau$$

En matemáticas, la **convolución es una operación entre dos funciones para producir una tercera función modificada**. Formalmente, representa la integral (o la sumatoria, en el caso discreto) del producto de una función con una versión trasladada y reflejada de la otra.

La primera función es la entrada (por ejemplo, una imagen cruda o un conjunto de datos espaciales).

La segunda función es el filtro o kernel, que es una pequeña matriz compuesta por parámetros o pesos que la red debe aprender.

Al resultado de esta operación combinada se lo denomina **mapa de características** (*feature map*).

1	1	1	0	0
0	1	1	1	0
0	0	1	1	1
0	0	1	1	0
0	1	1	0	0

Input

1	0	1
0	1	0
1	0	1

Filter / Kernel

En la literatura matemática y de procesamiento de señales existe una distinción formal entre ambas operaciones, pero **en el ámbito del aprendizaje automático (*machine learning*) ambas se consideran equivalentes y los términos se usan indistintamente.**

La diferencia técnica radica en lo siguiente:

- **Convolución:** Requiere que el filtro (o *kernel*) se invierta o "dé la vuelta" (se voltee a lo largo de su altura y anchura) antes de deslizarse sobre la imagen para realizar la multiplicación y la suma. Voltear el filtro es lo que le otorga a la convolución la **propiedad matemática de la conmutatividad**, lo cual es muy útil para escribir demostraciones analíticas, pero rara vez es importante para implementar una red neuronal.
- **Correlación cruzada:** Realiza exactamente el mismo proceso de deslizar el filtro sobre la entrada y calcular los productos, **pero sin voltear el kernel previamente**

En Pytorch, implementan la correlación cruzada discreta para evitar el paso extra de voltear el kernel, pero por convención la llaman "convolución".

Un **filtro** (también conocido comúnmente como **kernel**, *feature detector* o detector de propiedades) se define como una pequeña matriz de números (pesos) que se aplica sobre una imagen de entrada para extraer características y patrones significativos de la misma.

El mecanismo de "ventana deslizante" (Convolución): El filtro actúa como una especie de ventana que se va deslizando progresivamente sobre la imagen original, píxel por píxel. En cada posición, la red realiza una operación matemática (multiplicando y sumando los valores de los píxeles de la imagen por los valores numéricos del filtro) para obtener un nuevo valor.

1x1	1x0	1x1	0	0
0x0	1x1	1x0	1	0
0x1	0x0	1x1	1	1
0	0	1	1	0
0	1	1	0	0

4		

Generación de Mapas de Características: Al aplicar el filtro por toda la imagen, el resultado es una nueva matriz o imagen más pequeña llamada **mapa de características** (o *feature map*). Este mapa actúa como un indicador espacial que resalta en qué partes exactas de la imagen se activó o detectó el patrón que el filtro estaba buscando

Filtros

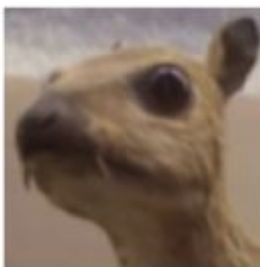
Input image



Kernel

$$\begin{pmatrix} \frac{1}{16} & \frac{1}{8} & \frac{1}{16} \\ \frac{1}{8} & \frac{1}{4} & \frac{1}{8} \\ \frac{1}{16} & \frac{1}{8} & \frac{1}{16} \end{pmatrix}$$

Feature map



Solution to the quiz above: The information diffuses nearly equally among all pixels; and this process will be stronger for neighboring pixels that differ more. This means that sharp edges will be smoothed out and information that is in one pixel, will diffuse and mix slightly with surrounding pixels. This kernel is known as a Gaussian blur or as Gaussian smoothing. [Continue](#)

[reading](#). Sources: [1](#) [2](#)

Input image



Convolution
Kernel

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Feature map



Jerarquía visual (de lo simple a lo complejo): En lugar de usar un único filtro, las capas de la red utilizan un "banco de filtros" para detectar múltiples cosas a la vez. Este proceso ocurre en cascada:

- En las primeras capas, los filtros detectan características muy básicas o de "bajo nivel", como cambios de contraste, bordes horizontales o verticales, líneas y colores.
- A medida que la red se hace más profunda, los nuevos filtros operan sobre los resultados anteriores, combinándolos para detectar patrones de "alto nivel" mucho más complejos y abstractos. Así, la red pasa de ver simples bordes a reconocer texturas, partes de objetos (como un ojo o la rueda de un coche) y, finalmente, el objeto en su totalidad.

Jerarquía visual en filtros

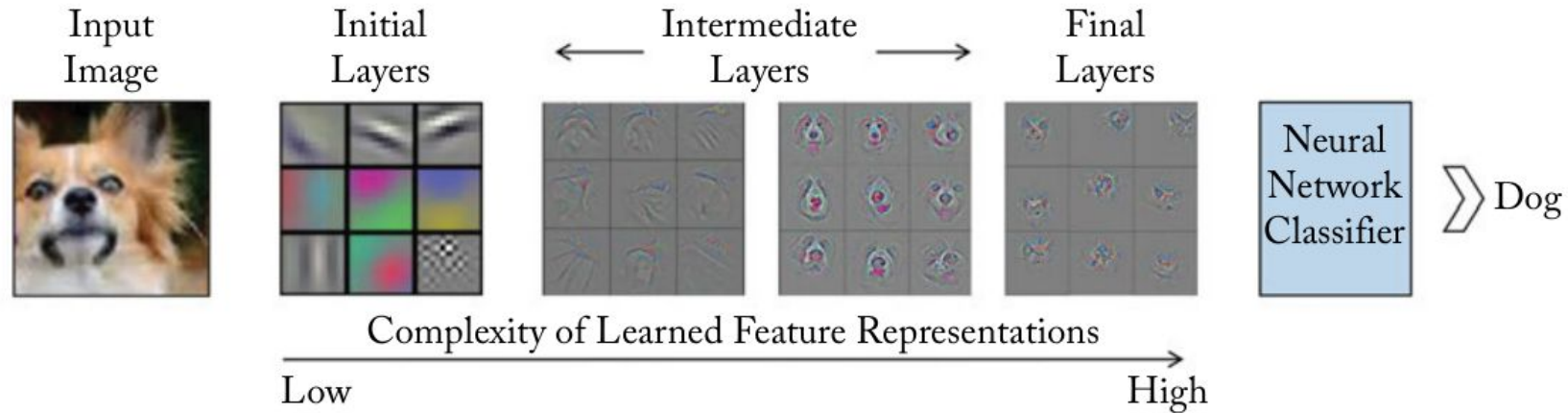


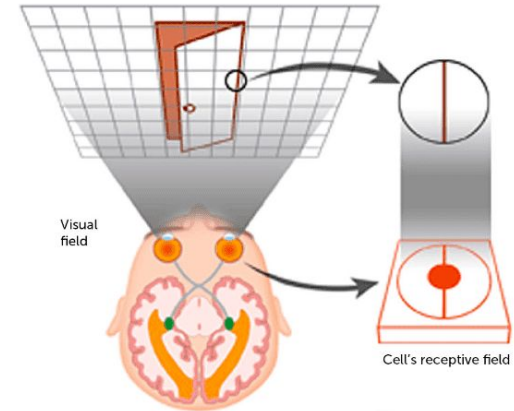
Figure 4.1: A CNN learns low-level features in the initial layers, followed by more complex intermediate and high-level feature representations which are used for a classification task. The feature visualizations are adopted from [Zeiler and Fergus \[2014\]](#).

Receptive field

En el contexto de las redes neuronales convolucionales (CNN), el *receptive field* se define como el área o región espacial específica de la imagen de entrada (o del mapa de características previo) sobre la cual se aplica y convoluciona un filtro para producir el valor de una única neurona o píxel en la capa de salida.

En términos sencillos, es la medida (altura y anchura) que define la extensión de la "ventana" espacial de los datos de entrada que un filtro puede observar, modificar o afectar en cada paso de la operación de convolución.

Este concepto está inspirado en la biología de la visión, específicamente en el comportamiento de las "células simples" de la corteza visual de los mamíferos, cuya actividad neuronal responde a los estímulos visuales que ocurren en una región muy pequeña y espacialmente localizada de su campo visual.



Receptive field, filtro y feature map



Capas convolucionales

El funcionamiento de esta capa se basa en el uso de **filtros o kernels**, que son pequeñas matrices de pesos. El proceso ocurre de la siguiente manera:

- El filtro se desliza a lo largo de la imagen de entrada píxel por píxel.
- En cada posición, el filtro cubre una pequeña área de la imagen llamada "campo receptivo" y realiza una operación matemática que consiste en multiplicar cada píxel por el peso correspondiente del filtro y sumar todos los resultados para producir un único valor central.

1x1	1x0	1x1	0	0
0x0	1x1	1x0	1	0
0x1	0x0	1x1	1	1
0	0	1	1	0
0	1	1	0	0

kernel

4		

feature map

- A medida que el filtro recorre toda la imagen repitiendo este cálculo, genera una nueva matriz bidimensional de salida conocida como **mapa de características** o mapa de activación. Si la imagen tiene múltiples canales de color (como RGB), el filtro actúa como un cubo tridimensional (ej. 3x3x3) y suma las contribuciones de todos los canales simultáneamente para generar el valor del mapa de características

Capas convolucionales

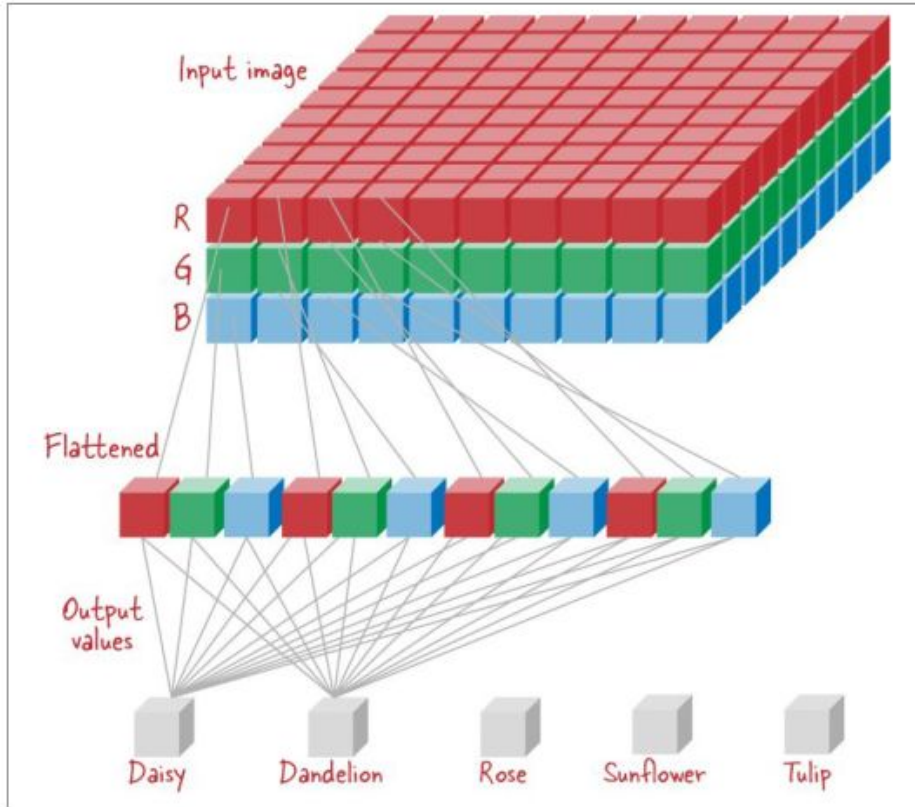
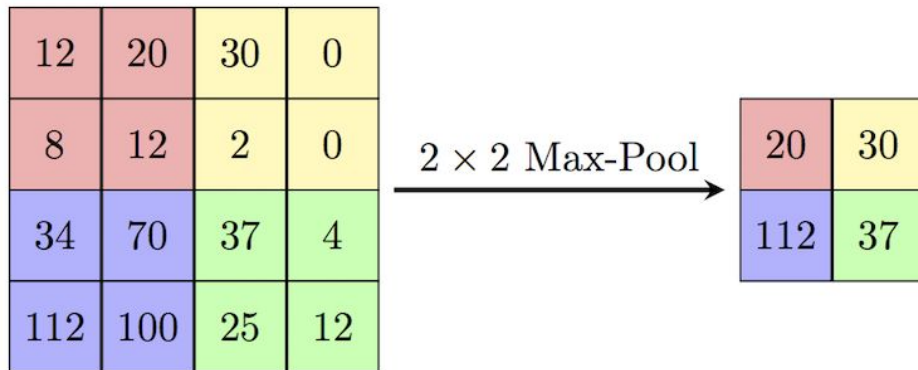


Figure 2-5. In the linear model, there are five outputs, one for each category; each of the output values is a weighted sum of the input pixel values.

Pooling

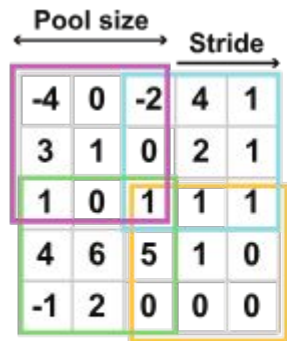
Las capas de *pooling* (también conocidas como capas de agrupación o submuestreo) operan sobre los mapas de características generados por las capas convolucionales previas. Su función principal es combinar las activaciones de las características reemplazando la salida en una cierta ubicación con una estadística resumida de las salidas cercanas.

Al aplicar *pooling*, las dimensiones espaciales (ancho y alto) del mapa de características se reducen, pero la profundidad (el número de filtros o canales) se mantiene exactamente igual. Puedes pensar en las capas de *pooling* como programas de compresión de imágenes: reducen drásticamente la resolución de la imagen mientras conservan sus características más importantes.



¿Por qué son necesarias estas capas?

- **Reducción de la complejidad y parámetros:** A medida que se añaden capas convolucionales, la cantidad de parámetros crece rápidamente. Las capas de *pooling* reducen la dimensionalidad de los datos que pasan a la siguiente capa, lo que disminuye el uso de memoria, el tiempo de entrenamiento y el costo de las operaciones matemáticas.
- **Invarianza a la traslación:** Ayudan a que la representación de la red sea aproximadamente invariante a pequeñas traslaciones o cambios. Esto significa que si un patrón en la imagen de entrada se desplaza ligeramente, los valores de salida agrupados apenas cambiarán, aportando robustez al modelo frente a variaciones de escala, pose o posición.

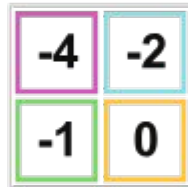


Features

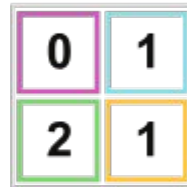
Max Pooling



Min Pooling



Average Pooling



Output

Al igual que en las convoluciones, el *pooling* utiliza una ventana que se desliza sobre el mapa de características. Sus hiperparámetros clave son:

- **Tamaño de la ventana (Pool size):** Define las dimensiones del bloque sobre el que se operará (por ejemplo, cuadrículas no superpuestas de 2x2).
- **Stride:** Determina cuántos píxeles salta la ventana en cada paso al deslizarse (por ejemplo, una zancada de 2).
- **Padding:** Al igual que en las convoluciones, el pooling tiene el hiperparámetro de Padding, aunque se usa mucho menos.

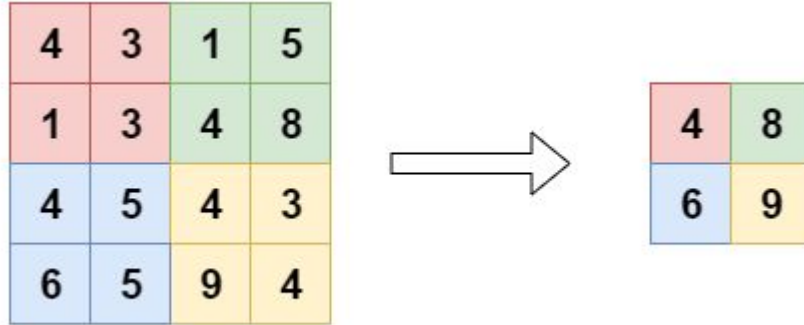
Valid (Sin padding): Es el valor por defecto. Si la ventana no entra perfectamente al final de la matriz, esos píxeles sobrantes se ignoran.

Same (Con padding): Añade filas/columnas de ceros en los bordes para asegurarse de que la ventana cubra absolutamente toda la matriz original.

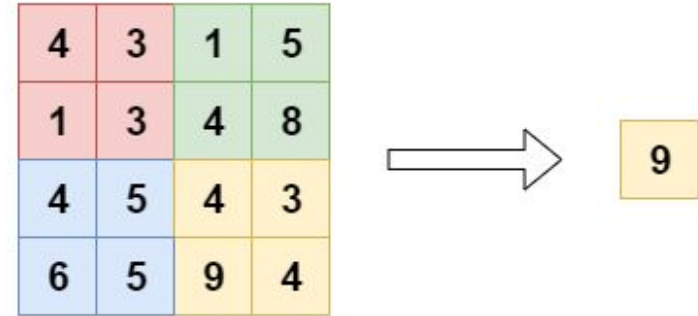
1. **Max Pooling**: Es el enfoque más utilizado en la actualidad. Esta operación simplemente selecciona el valor máximo dentro de la ventana y descarta el resto. La intuición es que al quedarse con el valor más alto, se asegura que las características más fuertes encontradas por los filtros (como bordes o líneas) sobrevivan al submuestreo, a expensas de las respuestas más débiles.
2. **Average Pooling**: Calcula el promedio aritmético de los píxeles dentro de la ventana. Aunque fue un enfoque común en los primeros modelos de redes neuronales, ha perdido popularidad frente al *max pooling*.
3. **Global Average Pooling**: Es una forma más extrema de reducción de dimensionalidad. En lugar de deslizar una pequeña ventana, calcula el promedio de *todos* los píxeles en un mapa de características entero. Su efecto principal es tomar una matriz tridimensional y aplastarla directamente para convertirla en un vector unidimensional.
4. **Adaptive Average Pooling**: Es un módulo versátil que toma la cuadrícula de activaciones y calcula su promedio para ajustarla a cualquier tamaño de destino que se requiera (casi siempre un tamaño de 1x1). Es muy popular en arquitecturas totalmente convolucionales.

Tipos de pooling

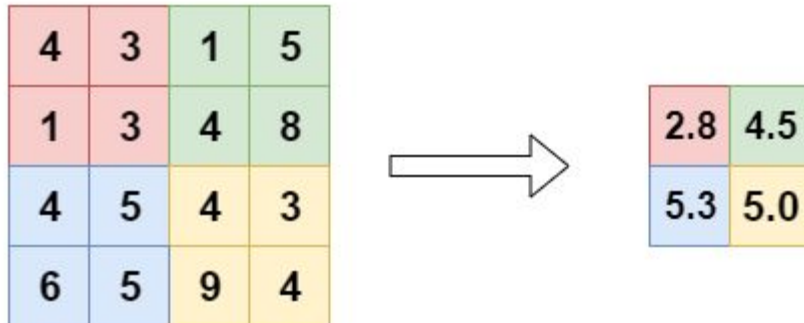
Max Pooling



GlobalAvg Pooling



Average Pooling

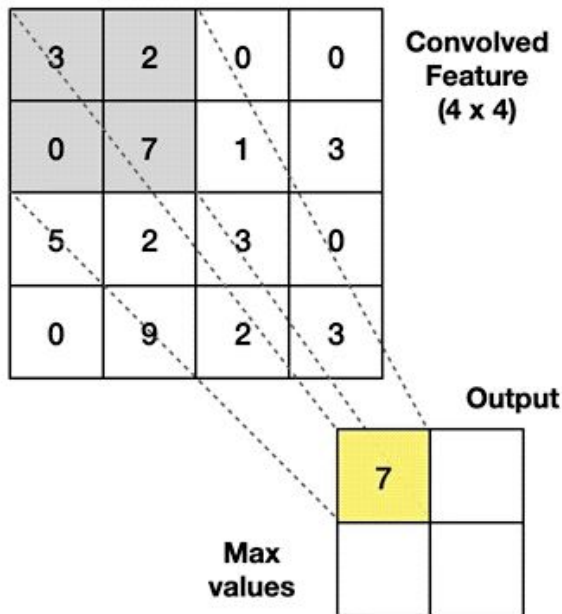


Pooling

Max Pooling

Take the **highest** value from the area covered by the kernel

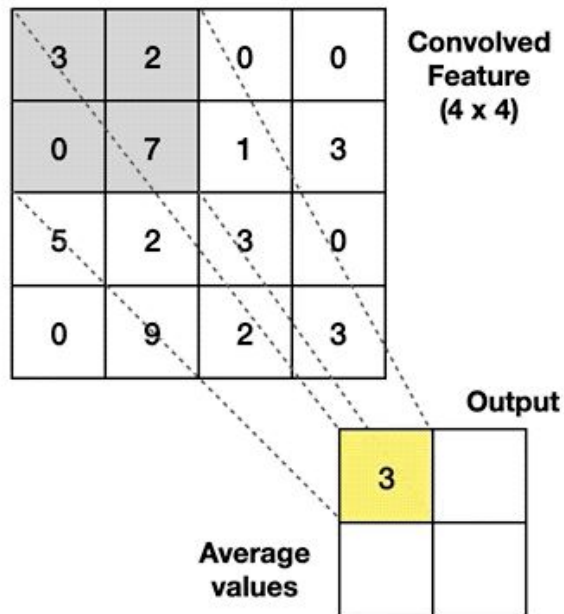
Example: Kernel of size 2×2 ; stride=(2,2)



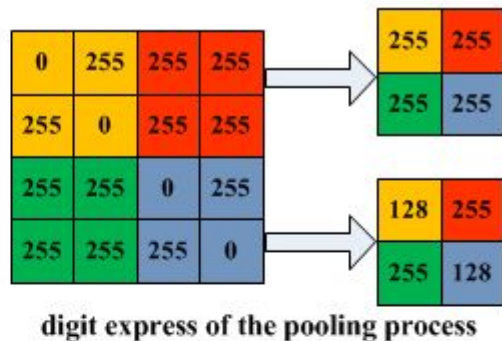
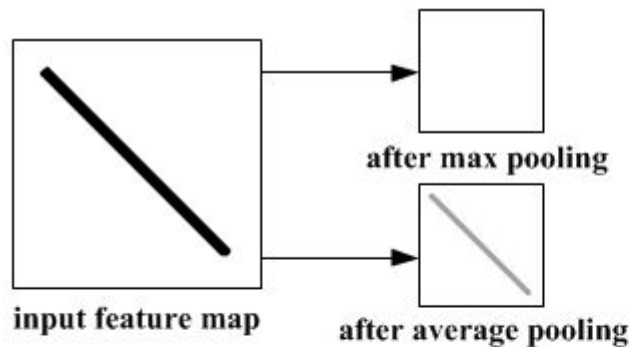
Average Pooling

Calculate the **average** value from the area covered by the kernel

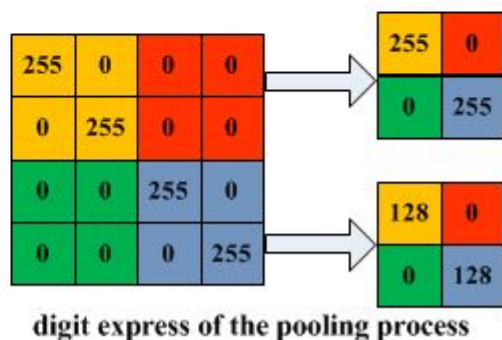
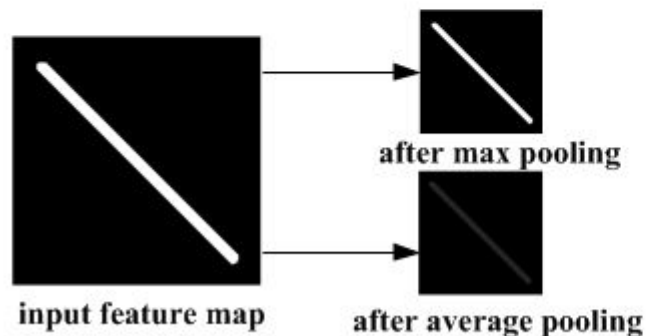
Example: Kernel of size 2×2 ; stride=(2,2)



Pooling



(a) Illustration of max pooling drawback



(b) Illustration of average pooling drawback

Tipos de Pooling

Original

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Max Pooling

6	8
14	16

Average pooling

3.5	5.5
11.5	13.5

Global Pooling

8.5

**Mixed
Pooling**

4.8	6.8
12.8	14.8

L2 Norm Pooling

8.1	11.7
23.4	27.3

Stochastic Pooling

6	4
9	11

Spectral Pooling

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

SPP Level 1

16

SPP Level 2

6	8
14	16

Stride

El **stride** (también conocido como zancada o paso) es un hiperparámetro clave en el diseño de redes neuronales convolucionales. Define **la cantidad de píxeles que el filtro (o *kernel*) salta o se desplaza cada vez que se mueve sobre la imagen de entrada.**

Dependiendo del valor que se le asigne al *stride*, el comportamiento y el tamaño de la salida de la capa convolucional cambian significativamente.

Strided convolution

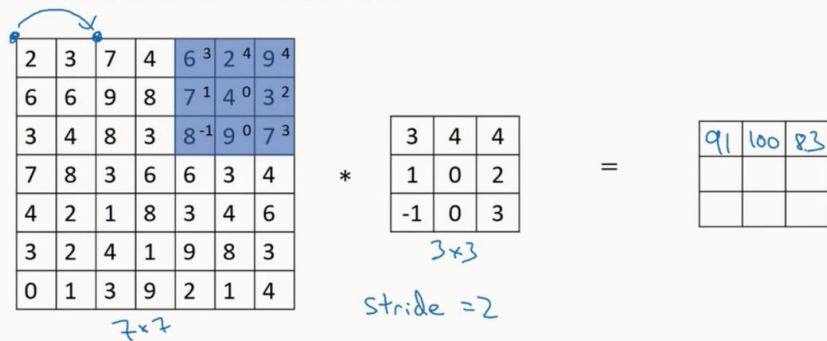


Diagram illustrating a strided convolution operation. The input is a 7x7 matrix, the kernel is a 3x3 matrix, and the output is a 3x3 matrix. The input matrix is annotated with a blue arrow indicating a stride of 2, and the output matrix is annotated with the handwritten values 91, 100, and 83.

2	3	7	4	6 ³	2 ⁴	9 ⁴
6	6	9	8	7 ¹	4 ⁰	3 ²
3	4	8	3	8 ⁻¹	9 ⁰	7 ³
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

7x7

3	4	4
1	0	2
-1	0	3

3x3
stride = 2

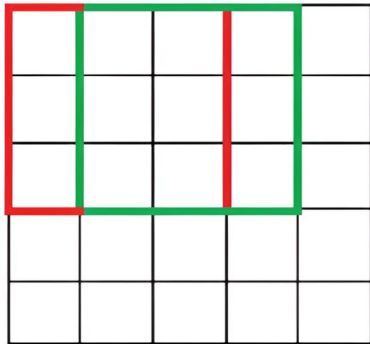
=

91	100	83

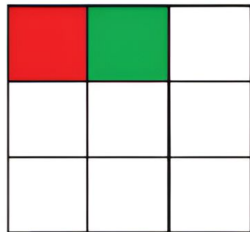
<https://medium.com/swlh/convolutional-neural-networks-part-2-padding-and-strided-convolutions-c63c25026ea>

Stride

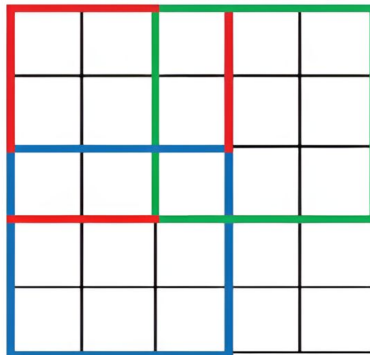
Convolution
with Stride=1



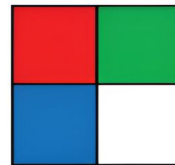
Output



Convolution
with Stride=2



Output



Stride de 1: Es el valor por defecto. El filtro se mueve de forma contigua, píxel por píxel. Al usar un *stride* de 1, el mapa de características resultante mantendrá aproximadamente el mismo ancho y alto que la imagen de entrada. Este enfoque es ideal cuando se desea añadir capas a la red para extraer más características sin alterar las dimensiones espaciales.

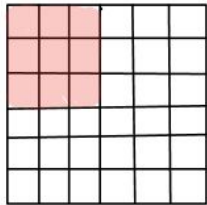
Stride de 2 (o mayor): El filtro avanza dando saltos, por ejemplo, moviéndose dos píxeles a la vez en cada paso. En la práctica, un *stride* de 2 es sumamente común, mientras que los saltos de 3 o más son muy raros.

Stride de 1

Input: 6x6x1

filter 3x3x1

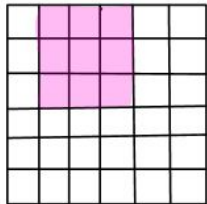
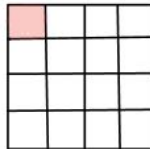
output 4x4x1



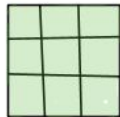
*



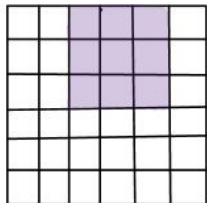
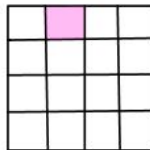
=



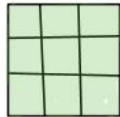
*



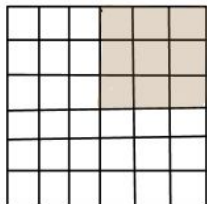
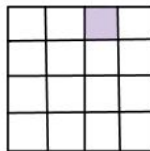
=



*



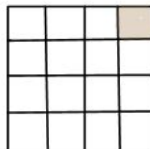
=



*



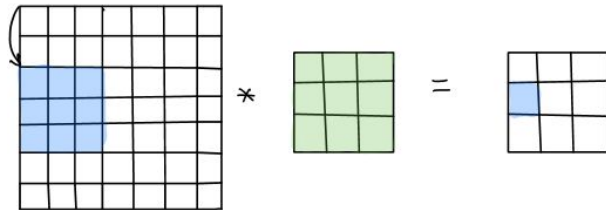
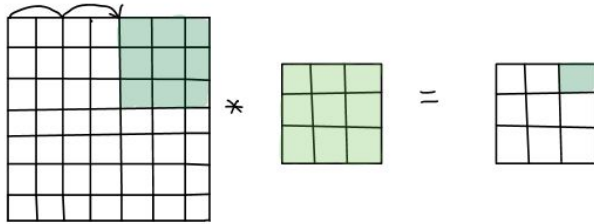
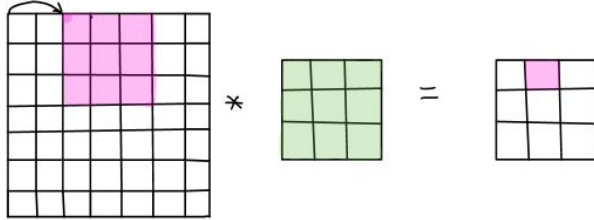
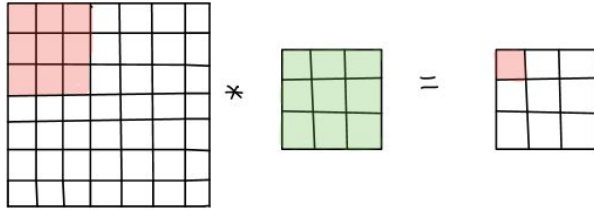
=



El filtro va recorriendo la data de entrada de a 1 pixel

Stride de 2

stride = 2

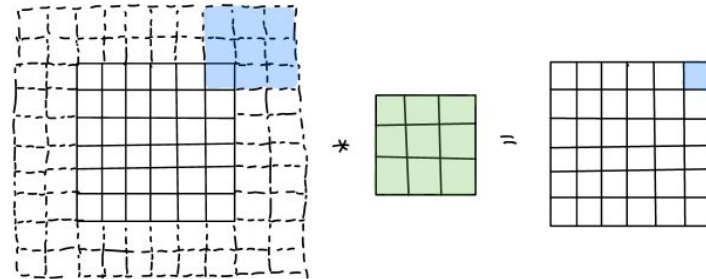
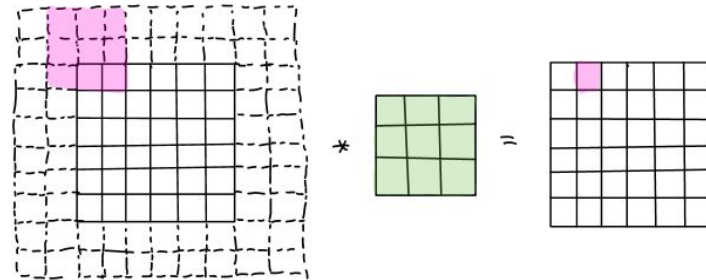
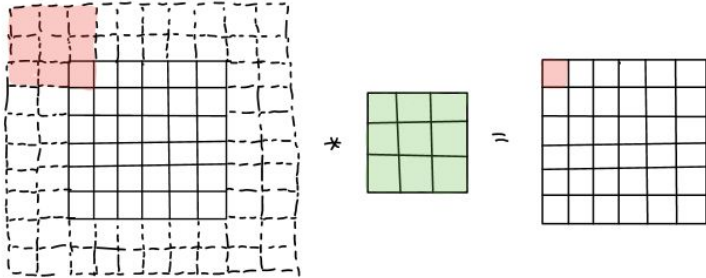


El filtro va recorriendo la data de
entrada de a 2 pixeles

El **padding** (frecuentemente llamado *zero-padding* o relleno de ceros) es una técnica que consiste en **agregar filas y columnas artificiales de píxeles alrededor de los bordes externos de una imagen o mapa de características** antes de aplicarle una operación de convolución. En la inmensa mayoría de los casos, a estos "píxeles fantasma" adicionales se les asigna el valor de cero.

Para entender por qué es tan importante, primero hay que ver el problema que resuelve: los **efectos de borde**. Cuando deslizas un filtro (o *kernel*) sobre una imagen, este necesita tener píxeles vecinos en todas las direcciones para realizar su cálculo. Si se está evaluando un píxel justo en la esquina o en el borde de la imagen, el filtro se sale de los límites. Si no se usa *padding*, el filtro simplemente ignorará esos bordes y solo operará donde quepa por completo. Como efecto secundario directo de esto, **la imagen de salida se "encoge" reduciendo su resolución espacial (ancho y alto) en cada convolución**

Padding



Distintos tipos de Padding

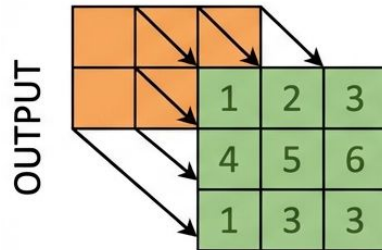
VALID PADDING

INPUT

5	2	3	4	5
7	6	7	4	8
3	4	5	6	6
1	2	3	4	5
0	1	3	2	5

FILTER

1	2	3
3	3	3
1	3	3



Output size is smaller:
 $(W - F + 1) \times (H - F + 1)$
 $= 3 \times 3$

SAME PADDING

0	0	0	0	0	0	0
0	5	2	3	4	5	0
0	7	6	7	4	8	0
0	3	4	5	6	6	0
0	1	2	3	4	5	0
0	0	1	3	2	5	0
0	0	0	0	0	0	0

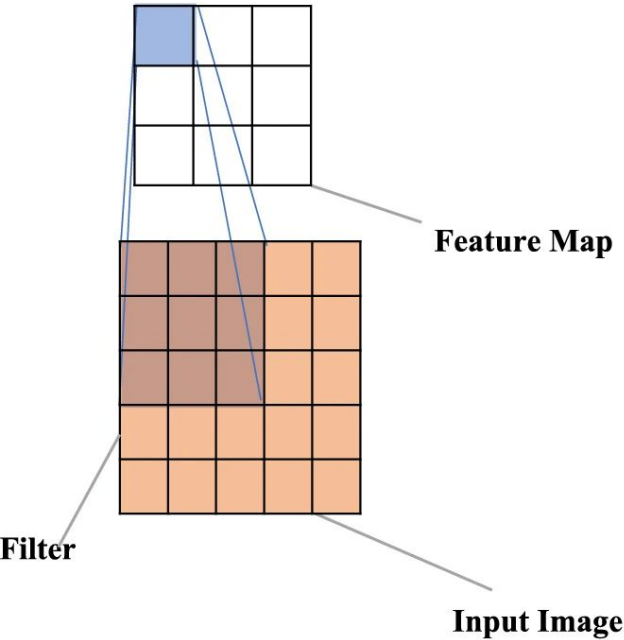
FILTER

1	2	3
3	3	3
1	3	3

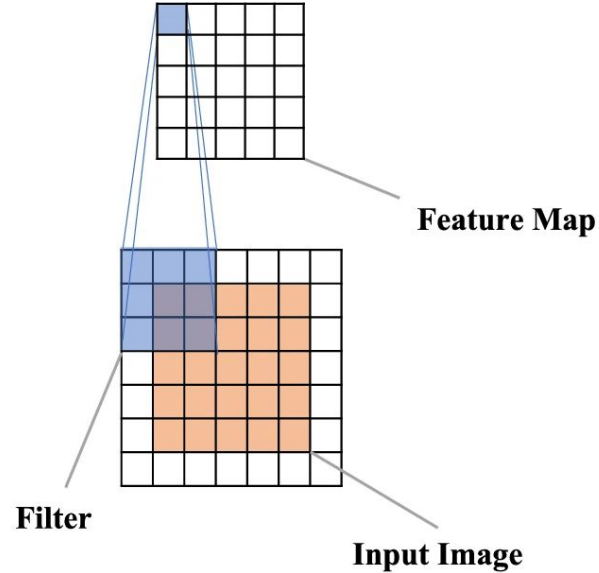
	5	2	6	2	5
	6	4	7	8	5
	8	3	3	6	7
	1	4	4	4	6
	5	5	3	3	5

Output size matches input size: 5x5.
Zeros added to maintain size.

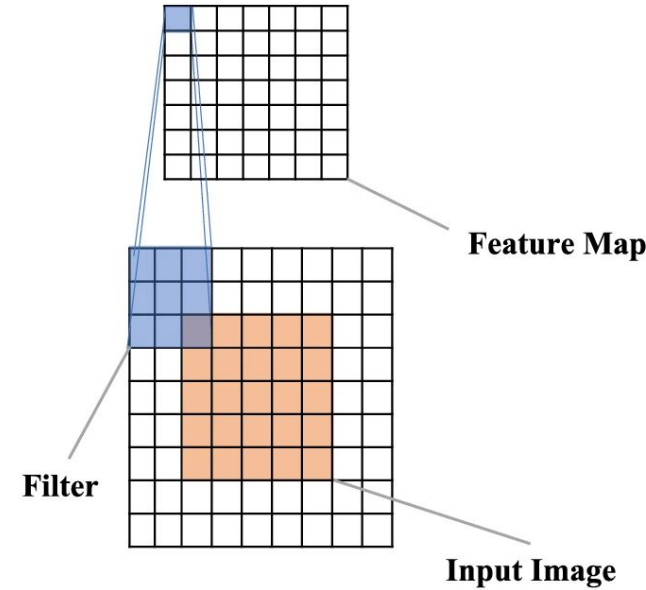
Distintos tipos de Padding



(a) Valid Padding



(b) Same Padding



(c) Full Padding

Distintos tipos de Padding

Table 2 Comparison of different padding types

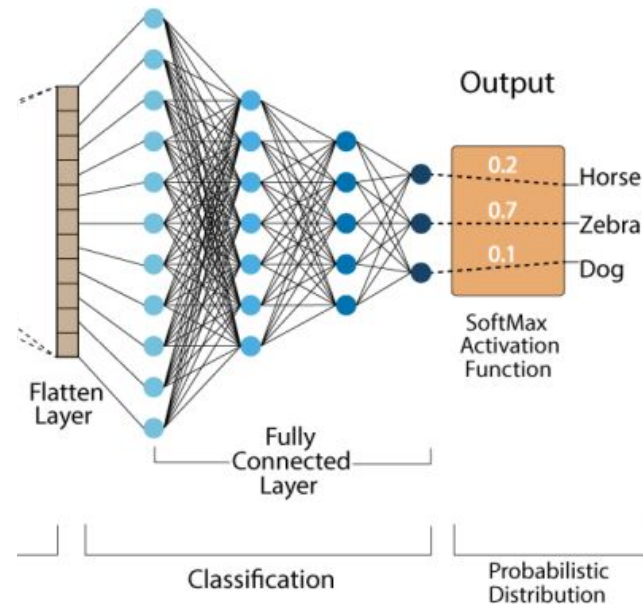
Padding type	Description	Mathematical formula for P	Impact on feature map size	Filter coverage	Computational cost	Use cases	Citations
Valid padding	No extra pixels are added to the input	$P=0$	Compared to the input feature map, the output feature map is smaller	Center regions only; borders ignored	Lowest	Suitable for tasks where edge information is irrelevant (e.g., central object detection)	[60]
Same padding	To guarantee that the output feature map size matches the input, padding is provided	$P = \frac{F-1}{2}$	Output feature map size remains unchanged	Full input, including borders	Moderate	Widely used in image recognition tasks where input–output size consistency is needed	[61]
Full padding	The padding ensures that the filter can cover the entire input, including borders	$P=F-1$	Output feature map is larger than the input	Full input extends coverage beyond the original input	Highest	Used in tasks requiring extended feature coverage, like object detection and segmentation	[58]

Capas densas

Las capas densas finales (también conocidas como capas totalmente conectadas o *fully connected*) son el componente de la red encargado de **realizar la clasificación final basándose en las características extraídas** por las capas anteriores. En términos prácticos, la sección final de la red funciona de manera idéntica a un perceptrón multicapa (MLP) tradicional.

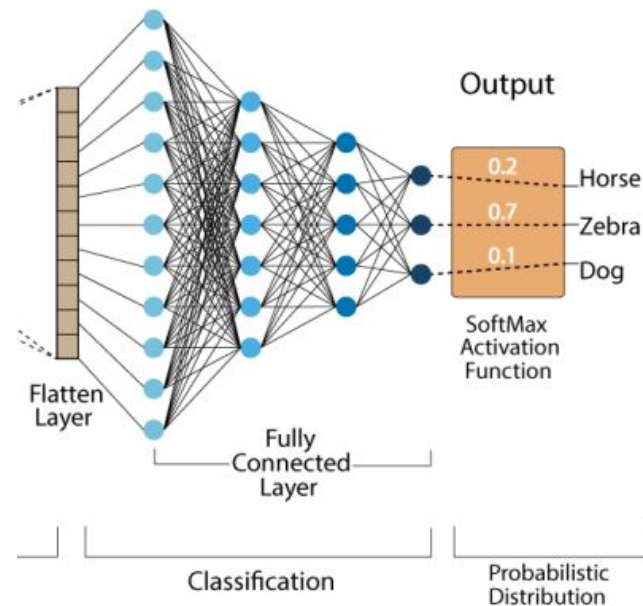
Las capas convolucionales y de *pooling* actúan como extractores de características, y su salida son mapas tridimensionales (ancho, alto y profundidad). Sin embargo, las capas densas solo aceptan vectores unidimensionales. Por lo tanto, el primer paso antes de entrar a esta etapa es **"aplanar" (operación *flatten*) los mapas de características**, convirtiéndolos en un solo vector largo.

Una vez aplanados los datos, las capas densas entran en acción con las siguientes características y propósitos clave

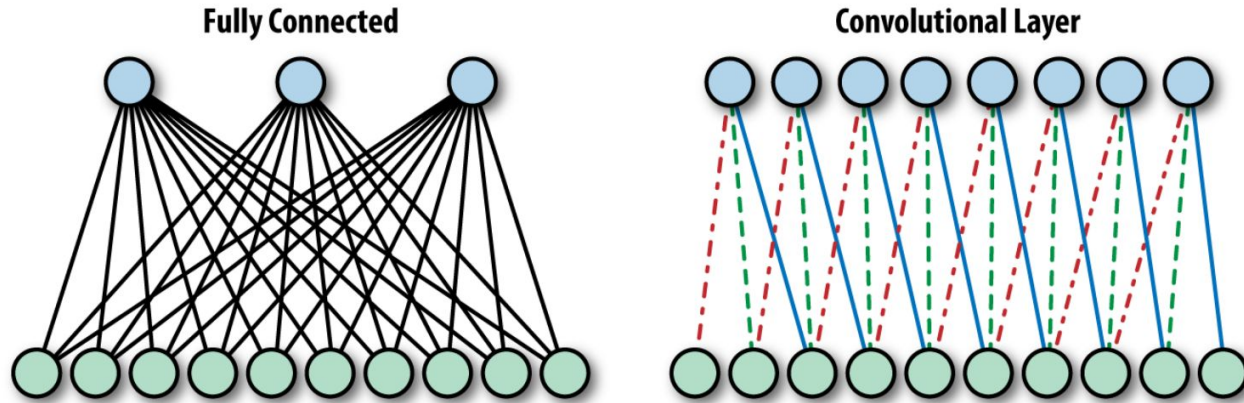


Capas densas

- **Toma de decisiones:** Las capas densas toman el vector que contiene todas las características de alto nivel detectadas por las convoluciones (como la presencia de una rueda, un ojo o una forma específica) y aprenden a combinarlas. Analizan estos rasgos para determinar de manera concluyente a qué categoría pertenece la imagen.
- **Generación de probabilidades:** La última capa densa está configurada para emitir la predicción final. Si el problema es de clasificación multiclase (por ejemplo, clasificar imágenes en 1000 categorías distintas o reconocer 10 dígitos), **se utiliza la función de activación *Softmax*, la cual convierte los valores de salida en una distribución de probabilidades que suman 1**. Si es una clasificación binaria (ej. perro o gato), se usa una capa de una sola neurona con activación *Sigmoid*, que arroja un valor entre 0 y 1.



Éxito de las redes convolucionales



Conectividad local (Interacciones dispersas): En lugar de conectar cada neurona de salida a todos los píxeles de entrada, las CNN operan sobre vecindarios locales. Utilizan filtros (o kernels) que son espacialmente más pequeños que la imagen de entrada, lo que significa que cada neurona solo se conecta a un pequeño subconjunto de la imagen, conocido como su "campo receptivo". Esto reduce drásticamente el número de parámetros a entrenar y los requisitos de memoria del modelo.

Éxito de redes convolucionales

Parámetros compartidos (Pesos compartidos/ weight sharing):

Los valores de los filtros no están estáticos en una sola posición, sino que el mismo filtro se desliza y se reutiliza a lo largo de toda la imagen. Esto significa que, en lugar de aprender un conjunto separado de parámetros para cada ubicación, la red aprende a detectar una característica específica independientemente de dónde aparezca.

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved
Feature

Invarianza a la traslación: Gracias a la operación de convolución y a los pesos compartidos, si un objeto o patrón se desplaza dentro de la imagen, la representación generada por la red también se desplaza en la misma medida. Esto las hace altamente eficientes y robustas ante variaciones espaciales comunes en el mundo real.

De forma más sencilla: la **invariancia a la traslación** significa que la red neuronal puede reconocer un patrón o un objeto **sin importar en qué parte de la imagen se encuentre**.

Por ejemplo, si la red aprende a identificar la cara de un gato en la esquina inferior derecha de una fotografía, será capaz de reconocer esa misma cara si en otra foto aparece en la esquina superior izquierda, o en el centro. Para la red, un gato sigue siendo un gato independientemente de si se ha movido (trasladado) dentro del encuadre.

Regularización

Todas las técnicas de regularización vistas, siguen vigentes:

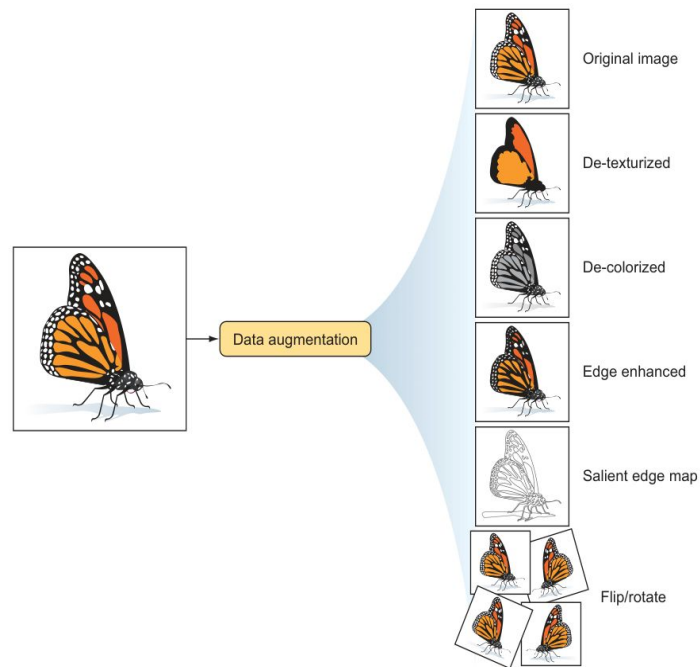
- Regularización L2 (weight decay), regularización L1.
- Early stopping
- Batch normalization

Y se agregan algunas técnicas específicas de redes CNN:

- Data augmentation
- DropConnect

Data Augmentation

Es una técnica casi universal en visión por computadora. Como es difícil y costoso recolectar más datos reales, esta técnica **genera nuevas imágenes de entrenamiento aplicando alteraciones sintéticas** a las imágenes originales, como rotaciones, recortes, acercamientos (*zoom*), cambios de brillo, volteos horizontales/verticales, y la adición de ruido. Al hacer esto, **el modelo nunca ve exactamente la misma imagen dos veces durante el entrenamiento**, lo que lo obliga a ser invariante a transformaciones espaciales y a generalizar mejor.



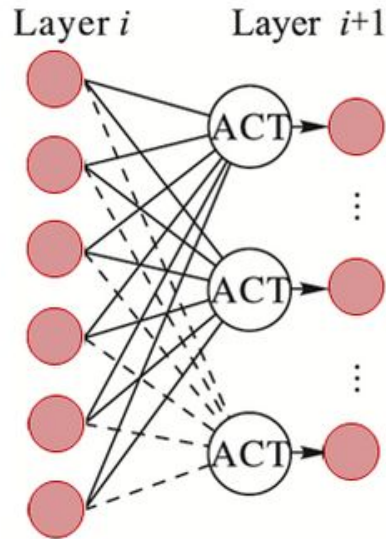
DropConnect es una técnica de regularización introducida en 2013 por Wan et al., diseñada para evitar el sobreajuste (*overfitting*) en las redes neuronales introduciendo un comportamiento estocástico durante el entrenamiento.

Se la considera una variante o un caso especial del *Dropout*, pero con una diferencia conceptual muy importante:

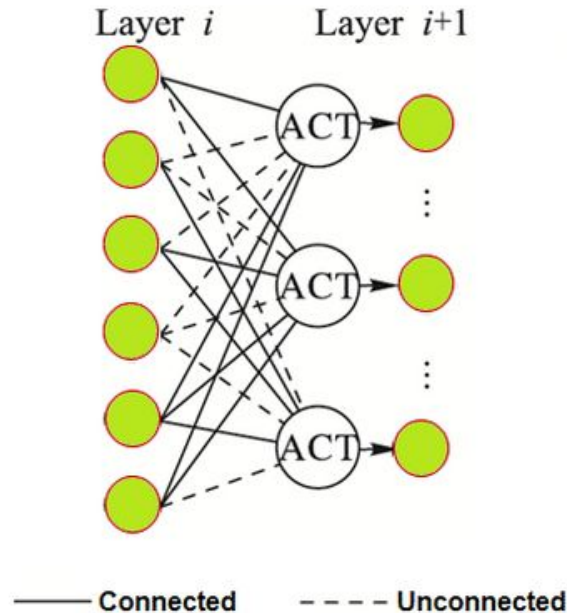
- **Mientras que el Dropout estándar** "apaga" aleatoriamente las **activaciones de las neuronas** (es decir, reduce el valor de salida de un nodo a cero),
- **El DropConnect** desactiva aleatoriamente **los pesos o las conexiones individuales** entre las neuronas.

Dropout vs DropConnect

A. Dropout



B. DropConnect



Mientras que en Dropout desactivamos neuronas enteras, en **DropConnect** **desactivamos algunas conexiones entre neuronas.**

Benchmarks para clasificación de imágenes

Un **benchmark** (o punto de referencia) en el ámbito del aprendizaje profundo y la visión por computadora es **un estándar de evaluación, compuesto típicamente por un conjunto de datos público y una tarea específica, que la comunidad investigadora utiliza para medir, comparar y validar el rendimiento de diferentes algoritmos o arquitecturas.**

Estos benchmarks frecuentemente tienen su origen en competencias académicas o desafíos de software. Un ejemplo representativo es el *ImageNet Large Scale Visual Recognition Challenge* (ILSVRC), utilizado como benchmark para comparar el desempeño de distintas redes en la clasificación y detección de objetos.

Las principales características y propósitos de los benchmarks incluyen:

- **Establecer el estado del arte**
- **Evaluación multidimensional**
- **Comparación justa y controlada**

MNIST

La **base de datos MNIST** (por sus siglas en inglés, *Modified National Institute of Standards and Technology database*) es una extensa colección de base de datos que se utiliza ampliamente para el entrenamiento de diversos sistemas de procesamiento de imágenes.

La base de datos MNIST consta de 60.000 imágenes de entrenamiento y 10.000 imágenes de prueba.



Fashion-MNIST

Creado como un reemplazo directo y más desafiante para MNIST. Mantiene el mismo tamaño y formato (imágenes en escala de grises de 28x28), pero contiene 10 categorías de prendas de vestir y calzado, lo que presenta patrones visuales más complejos.



CIFAR

CIFAR-10 y CIFAR-100: Constan de 60.000 imágenes a color de baja resolución (32x32 píxeles) divididas en 10 o 100 clases, respectivamente. Son *benchmarks* indispensables para la experimentación rápida, la enseñanza y el análisis del comportamiento de generalización de los modelos antes de escalarlos a problemas masivos

airplane



automobile



bird



cat



deer



dog



frog



horse



ship

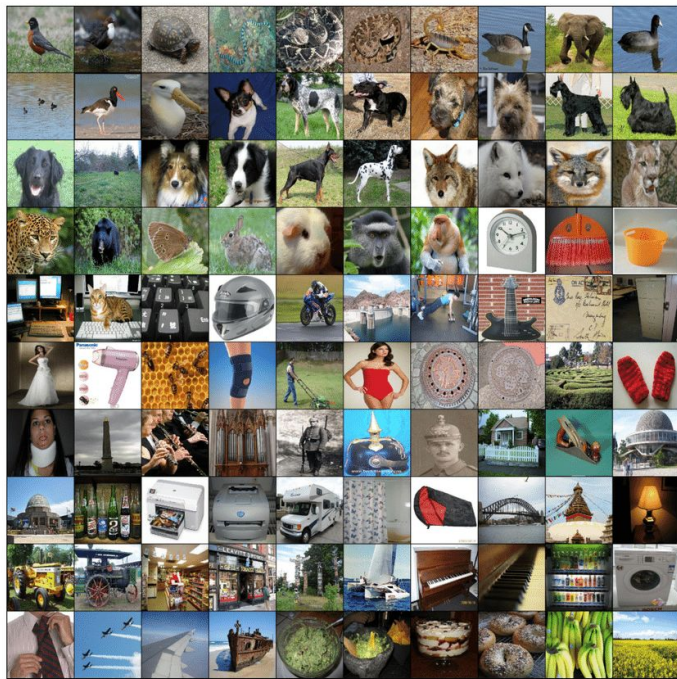


truck



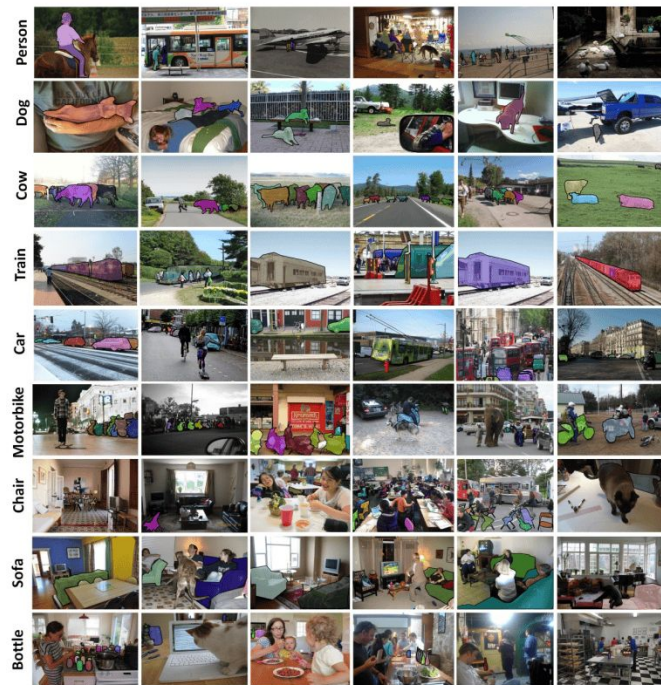
ImageNet

ImageNet / ILSVRC: El *ImageNet Large Scale Visual Recognition Challenge* (ILSVRC) es, casi con total seguridad, el **benchmark** más influyente en la historia moderna del **deep learning**. Al proveer más de 1.2 millones de imágenes etiquetadas distribuidas en 1000 categorías complejas, no solo estandarizó una métrica de evaluación global, sino que brindó por primera vez el volumen de datos necesario para entrenar redes profundas exitosamente



MS COCO

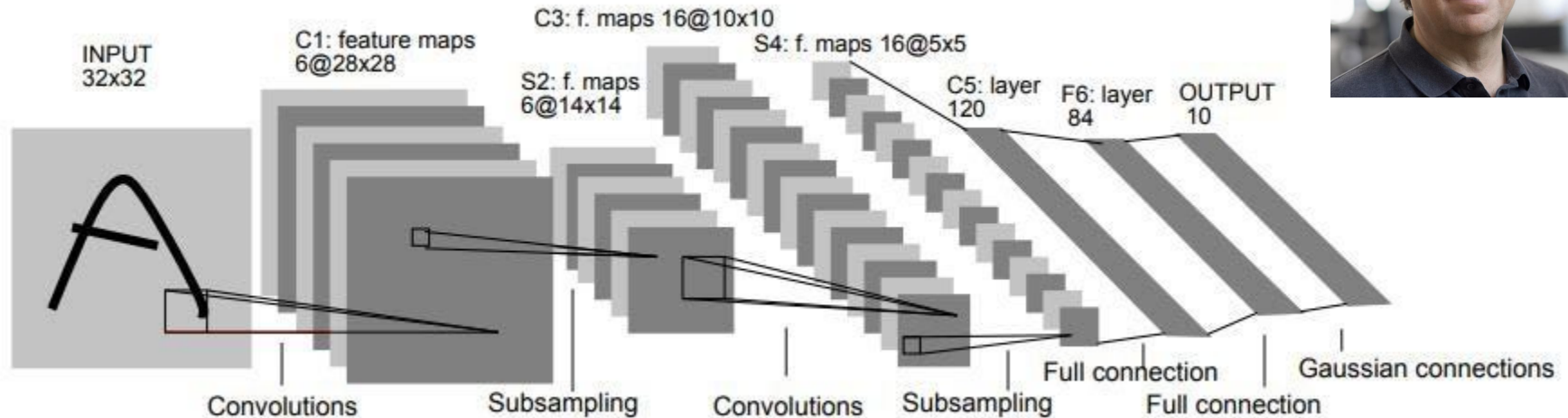
MS COCO (*Microsoft Common Objects in Context*): Es el *benchmark* predominante en la actualidad para tareas visuales complejas. Contiene más de 300.000 imágenes (más de 200.000 etiquetadas exhaustivamente) que engloban 1.5 millones de instancias de objetos en 80 categorías distintas. A diferencia de ImageNet (que se centra en clasificar una imagen entera), COCO se utiliza para evaluar modelos en tareas avanzadas como **detección de objetos (cajas delimitadoras)**, **segmentación de instancias a nivel de píxel y generación de subtítulos descriptivos (*image captioning*)**.



Arquitecturas relevantes en computer vision

LeNet-5

Desarrollado por Yann LeCun



LeNet-5 (1998) Desarrollada por Yann LeCun y su equipo, es **una de las redes convolucionales pioneras**. Se diseñó en un principio para tareas de reconocimiento de documentos, logrando un gran éxito clasificando dígitos escritos a mano (el famoso conjunto de datos MNIST).

- **Arquitectura:** Es una red relativamente pequeña compuesta por **5 capas con pesos entrenables** (tres capas convolucionales y dos totalmente conectadas).
- **Aporte histórico:** Estableció el patrón clásico de diseño que siguen la mayoría de las redes: extraer características alternando capas convolucionales y capas de *pooling* (submuestreo), para luego aplanar los datos y pasarlos por capas densas de clasificación.
- **Diferencias con la actualidad:** A diferencia de las redes modernas, LeNet-5 utilizaba *Average Pooling* (agrupación promedio) en lugar de *Max Pooling*, y empleaba **funciones de activación Tanh o sigmoideas**, ya que la función ReLU aún no era el estándar en el aprendizaje profundo.

Gradient-Based Learning Applied to Document Recognition

Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner

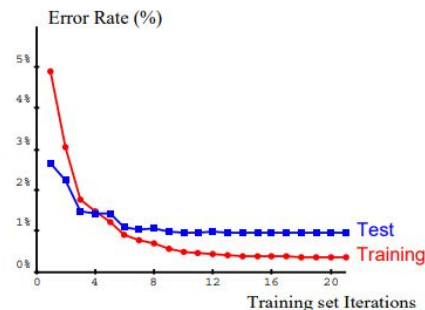


Fig. 5. Training and test error of LeNet-5 as a function of the number of passes through the 60,000 pattern training set (without distortions). The average training error is measured on-the-fly as training proceeds. This explains why the training error appears to be larger than the test error. Convergence is attained after 10 to 12 passes through the training set.

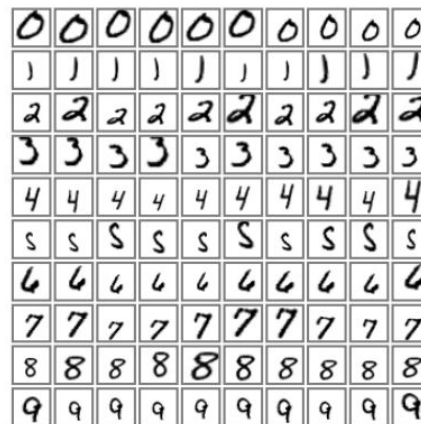


Fig. 7. Examples of distortions of ten training patterns.

Abstract

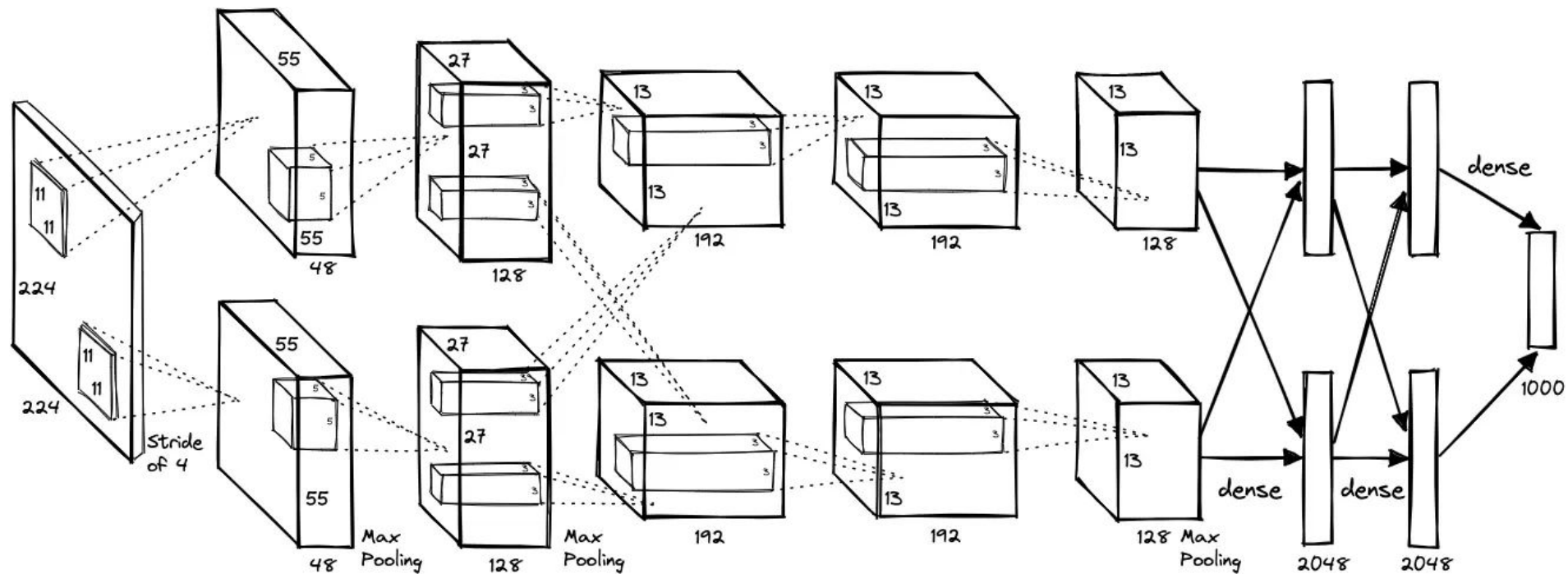
Multilayer Neural Networks trained with the backpropagation algorithm constitute the best example of a successful Gradient-Based Learning technique. Given an appropriate network architecture, Gradient-Based Learning algorithms can be used to synthesize a complex decision surface that can classify high-dimensional patterns such as handwritten characters, with minimal preprocessing. This paper reviews various methods applied to handwritten character recognition and compares them on a standard handwritten digit recognition task. Convolutional Neural Networks, that are specifically designed to deal with the variability of 2D shapes, are shown to outperform all other techniques.

Real-life document recognition systems are composed of multiple modules including field extraction, segmentation, recognition, and language modeling. A new learning paradigm, called Graph Transformer Networks (GTN), allows such multi-module systems to be trained globally using Gradient-Based methods so as to minimize an overall performance measure.

Two systems for on-line handwriting recognition are described. Experiments demonstrate the advantage of global training, and the flexibility of Graph Transformer Networks.

A Graph Transformer Network for reading bank check is also described. It uses Convolutional Neural Network character recognizers combined with global training techniques to provides record accuracy on business and personal checks. It is deployed commercially and reads several million checks per day.

Keywords— Neural Networks, OCR, Document Recognition, Machine Learning, Gradient-Based Learning, Convolutional Neural Networks, Graph Transformer Networks, Finite State Transducers.



AlexNet (2012) Creada por Alex Krizhevsky, Ilya Sutskever y Geoffrey Hinton, esta red marcó un **punto de inflexión histórico** para la inteligencia artificial. En 2012, ganó la competencia ImageNet (ILSVRC) por un margen enorme, reduciendo la tasa de error del 26.2% al 15.3%, lo que demostró al mundo que el *deep learning* era el camino a seguir para la visión por computadora.

- **Arquitectura:** Es significativamente más profunda y masiva que LeNet. Consta de **8 capas con pesos** (cinco convolucionales y tres densas finales), agrupando un total de **60 millones de parámetros** y 650.000 neuronas. Debido a este tamaño, sus creadores tuvieron que dividir su entrenamiento en múltiples tarjetas gráficas (GPUs) durante casi una semana.
- **Innovaciones tecnológicas:** Popularizó el uso de la **función de activación ReLU**, la cual entrena mucho más rápido que *Tanh* y ayuda a evitar el problema del desvanecimiento del gradiente. También fue pionera en implementar el uso de **capas de Dropout** y técnicas de **aumento de datos (Data Augmentation)** como herramientas críticas para evitar que un modelo tan grande memorizara los datos de entrenamiento (*overfitting*). Utilizaba filtros de tamaño grande en sus primeras etapas, como kernels de 11x11 y 5x5

ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

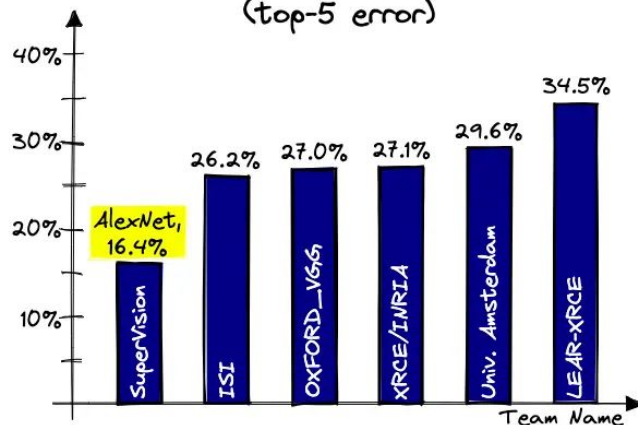
Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

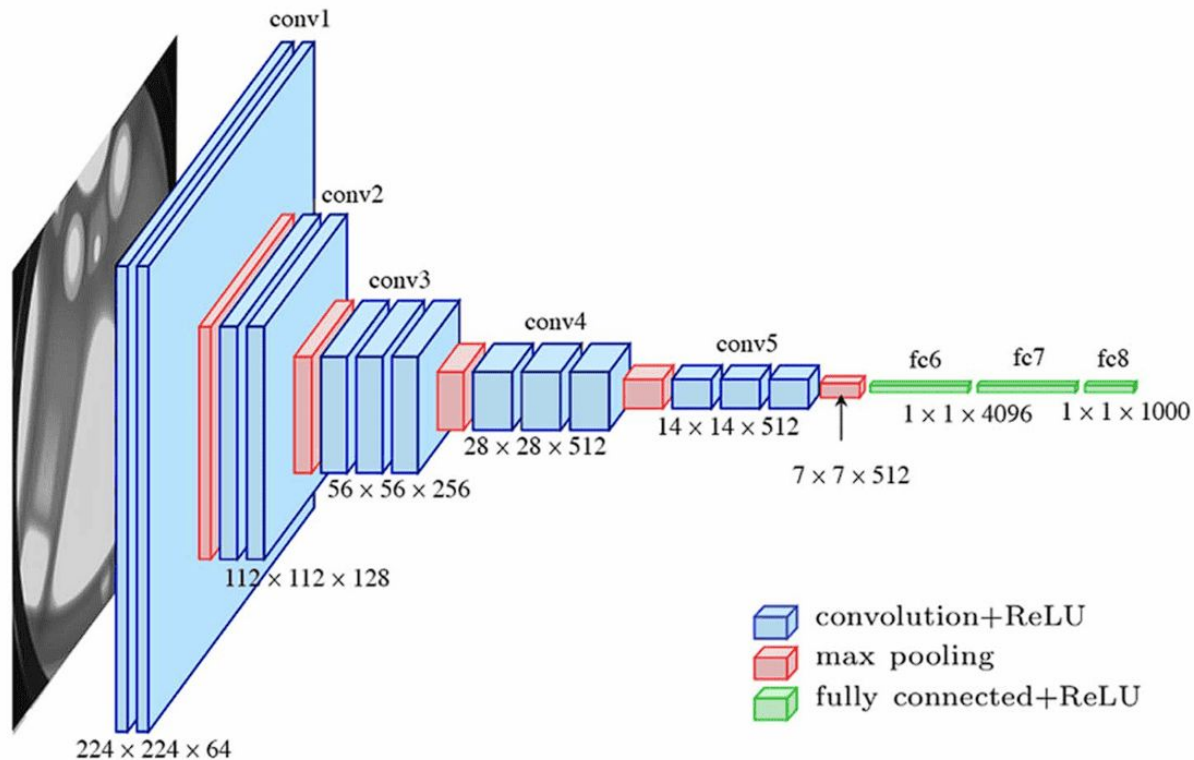
Abstract

We trained a large, deep convolutional neural network to classify the 1.2 million high-resolution images in the ImageNet ILSVRC-2010 contest into the 1000 different classes. On the test data, we achieved top-1 and top-5 error rates of 37.5% and 17.0% which is considerably better than the previous state-of-the-art. The neural network, which has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax. To make training faster, we used non-saturating neurons and a very efficient GPU implementation of the convolution operation. To reduce overfitting in the fully-connected layers we employed a recently-developed regularization method called “dropout” that proved to be very effective. We also entered a variant of this model in the ILSVRC-2012 competition and achieved a winning top-5 test error rate of 15.3%, compared to 26.2% achieved by the second-best entry.

2012 ImageNet Challenge
(top-5 error)



VGGNet



VGGNet / VGG16 (2014) Desarrollada por el *Visual Geometry Group* de la Universidad de Oxford (Simonyan y Zisserman), su principal objetivo fue demostrar que **la profundidad de la red es un factor crítico** para mejorar el rendimiento.

- **Arquitectura:** Sus configuraciones más utilizadas tienen **16 o 19 capas con pesos** (VGG16 y VGG19), lo que la hizo una red sumamente profunda para su época, alcanzando unos 138 millones de parámetros en su versión VGG16.
- **Diseño estrictamente uniforme:** La mayor innovación de VGGNet fue **eliminar por completo los filtros de convolución grandes** (como los de 11x11 de AlexNet). En su lugar, instauró la regla de usar **únicamente pequeños filtros de 3x3** que avanzan de a un píxel (*stride* de 1), intercalados esporádicamente con capas de *max pooling* de 2x2 para reducir la resolución.
- **Por qué fue revolucionaria:** Los autores demostraron que apilar múltiples capas con pequeños filtros de 3x3 otorga a la red el mismo "campo receptivo" (la cantidad de imagen que puede observar) que un filtro grande, pero con la ventaja de tener **menos parámetros matemáticos y de incorporar más funciones no lineales (ReLU)** en cada paso. Esto permitió crear modelos mucho más profundos que aprenden características más complejas, todo con una arquitectura muy sencilla y fácil de comprender

VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION

Karen Simonyan* & Andrew Zisserman⁺

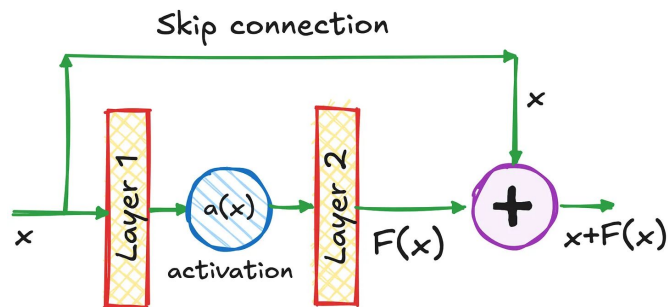
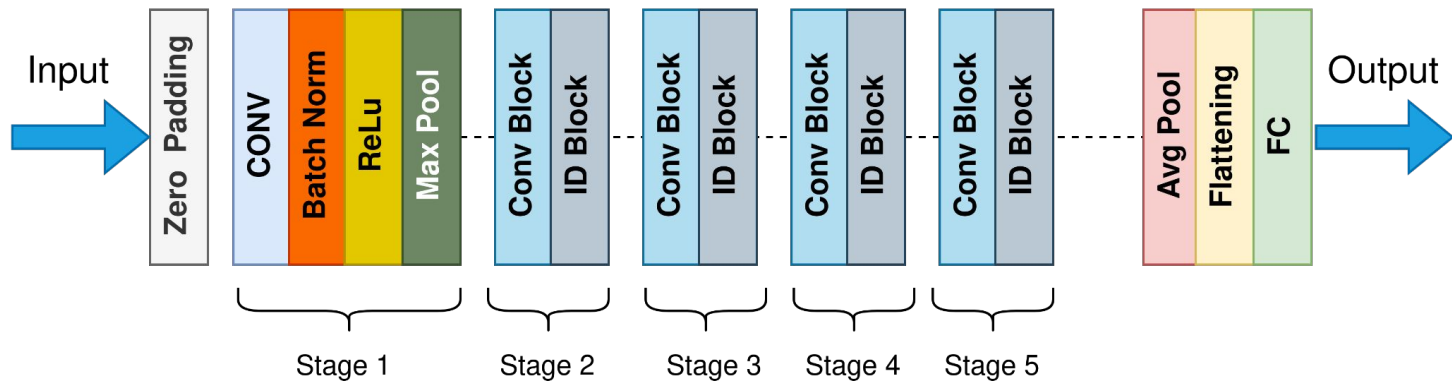
Visual Geometry Group, Department of Engineering Science, University of Oxford
{karen,az}@robots.ox.ac.uk

ABSTRACT

In this work we investigate the effect of the convolutional network depth on its accuracy in the large-scale image recognition setting. Our main contribution is a thorough evaluation of networks of increasing depth using an architecture with very small (3×3) convolution filters, which shows that a significant improvement on the prior-art configurations can be achieved by pushing the depth to 16–19 weight layers. These findings were the basis of our ImageNet Challenge 2014 submission, where our team secured the first and the second places in the localisation and classification tracks respectively. We also show that our representations generalise well to other datasets, where they achieve state-of-the-art results. We have made our two best-performing ConvNet models publicly available to facilitate further research on the use of deep visual representations in computer vision.

ResNet

ResNet50 Model Architecture



Para ver la arquitectura de ResNet-34:

<https://arxiv.org/pdf/1512.03385>

ResNet (Residual Networks, 2015) Es, con diferencia, la arquitectura más utilizada en la actualidad y representa un hito fundamental en el aprendizaje profundo.

- **El problema que resolvió:** A medida que los investigadores intentaban crear redes cada vez más profundas para mejorar el rendimiento, se encontraron con el problema del "desvanecimiento del gradiente" (*vanishing gradient*), donde la red dejaba de aprender e incluso empeoraba su rendimiento.
- **La innovación:** Introdujo las **conexiones residuales** o "**skip connections**". En lugar de pasar la información estrictamente de una capa a la siguiente, estas conexiones permiten que la entrada original de un bloque se sume directamente a su salida, saltándose capas intermedias. Esto estabiliza el flujo de los gradientes, permitiendo entrenar redes extremadamente profundas (como ResNet-50 o ResNet-152) de forma estable y rápida.

Deep Residual Learning for Image Recognition

Abstract

Deeper neural networks are more difficult to train. We present a residual learning framework to ease the training of networks that are substantially deeper than those used previously. We explicitly reformulate the layers as learning residual functions with reference to the layer inputs, instead of learning unreferenced functions. We provide comprehensive empirical evidence showing that these residual networks are easier to optimize, and can gain accuracy from considerably increased depth. On the ImageNet dataset we evaluate residual nets with a depth of up to 152 layers— $8\times$ deeper than VGG nets [41] but still having lower complexity. An ensemble of these residual nets achieves 3.57% error on the ImageNet test set. This result won the 1st place on the ILSVRC 2015 classification task. We also present analysis on CIFAR-10 with 100 and 1000 layers.

The depth of representations is of central importance for many visual recognition tasks. Solely due to our extremely deep representations, we obtain a 28% relative improvement on the COCO object detection dataset. Deep residual nets are foundations of our submissions to ILSVRC & COCO 2015 competitions¹, where we also won the 1st places on the tasks of ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation.

Kaiming He

Xiangyu Zhang

Shaoqing Ren

Jian Sun

Microsoft Research

{kahe, v-xiangz, v-shren, jiansun}@microsoft.com

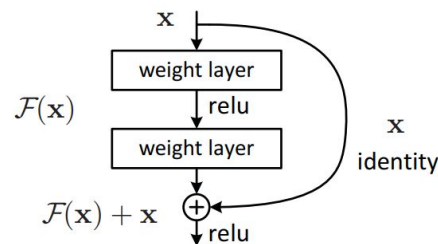
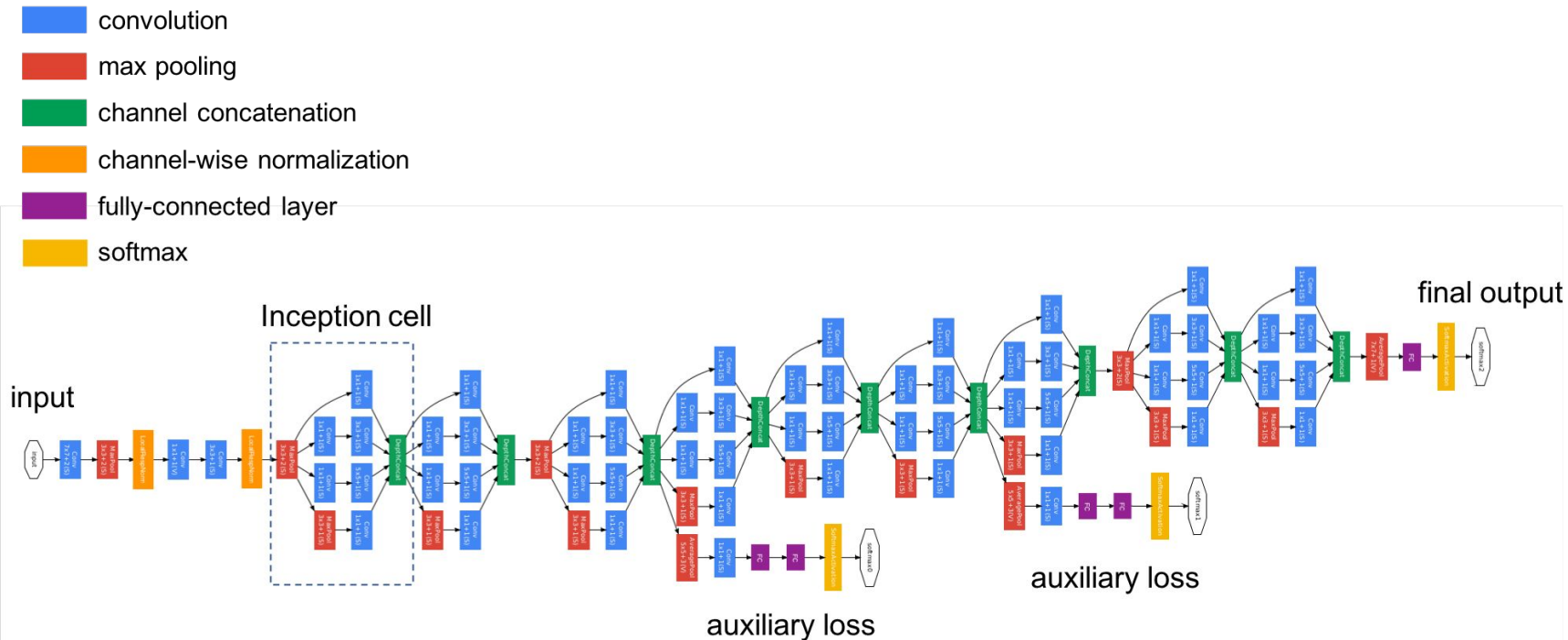


Figure 2. Residual learning: a building block.

<https://arxiv.org/pdf/1512.03385>



GoogLeNet / Arquitectura Inception (2014) Mientras VGG apostó por la profundidad mediante el apilamiento simple, GoogLeNet apostó por la complejidad interna de las capas.

- **La innovación:** En lugar de apilar capas convolucionales de forma tradicional, Szegedy y su equipo introdujeron el **módulo Inception**.
- **Cómo funciona:** Este módulo procesa la misma entrada utilizando múltiples operaciones en paralelo (filtros de 1x1, 3x3, 5x5 y una capa de *max-pooling*). Luego, combina todos estos mapas de características en una sola salida. Esto permite a la red extraer patrones a múltiples escalas y niveles de detalle de manera simultánea, logrando un gran rendimiento con un uso muy eficiente de los recursos computacionales.

Going deeper with convolutions

Christian Szegedy

Google Inc.

Wei Liu

University of North Carolina, Chapel Hill

Yangqing Jia

Google Inc.

Pierre Sermanet

Google Inc.

Scott Reed

University of Michigan

Dragomir Anguelov

Google Inc.

Dumitru Erhan

Google Inc.

Vincent Vanhoucke

Google Inc.

Andrew Rabinovich

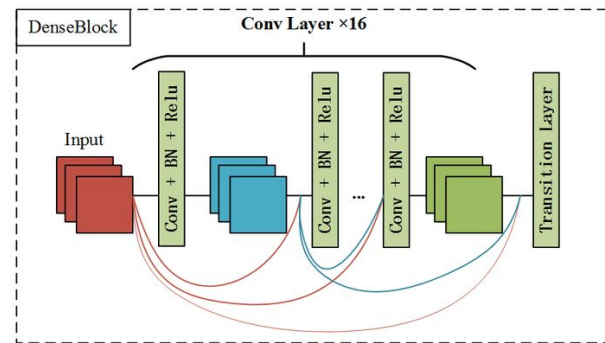
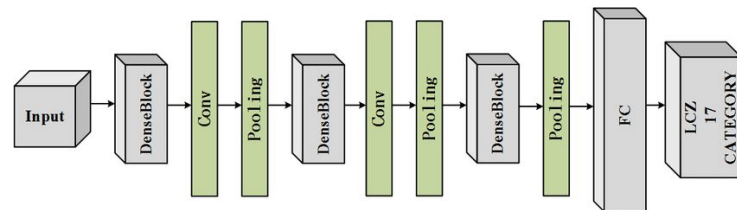
Google Inc.

Abstract

We propose a deep convolutional neural network architecture codenamed Inception, which was responsible for setting the new state of the art for classification and detection in the ImageNet Large-Scale Visual Recognition Challenge 2014 (ILSVRC14). The main hallmark of this architecture is the improved utilization of the computing resources inside the network. This was achieved by a carefully crafted design that allows for increasing the depth and width of the network while keeping the computational budget constant. To optimize quality, the architectural decisions were based on the Hebbian principle and the intuition of multi-scale processing. One particular incarnation used in our submission for ILSVRC14 is called GoogLeNet, a 22 layers deep network, the quality of which is assessed in the context of classification and detection.

DenseNet

La arquitectura **DenseNet (2018)** es un tipo de red neuronal convolucional en la que cada capa se conecta directamente con todas las capas subsiguientes en la red, recibiendo como entrada los mapas de características concatenados de todas las capas anteriores. Su principal mejora con respecto a arquitecturas convolucionales anteriores radica en fomentar la reutilización de características ("feature reuse"). Al concatenar la información en lugar de sumarla, DenseNet garantiza un flujo máximo de información y gradientes a lo largo de toda la red, lo que mitiga el problema del desvanecimiento del gradiente. Esta conectividad densa evita la necesidad de volver a aprender mapas de características redundantes, permitiendo que la red sea mucho más compacta, fácil de entrenar y altamente eficiente, ya que logra un rendimiento de vanguardia requiriendo una cantidad sustancialmente menor de parámetros.



Densely Connected Convolutional Networks

Gao Huang*
Cornell University
gh349@cornell.edu

Zhuang Liu*
Tsinghua University
liuzhuang13@mails.tsinghua.edu.cn

Laurens van der Maaten
Facebook AI Research
lvdmaaten@fb.com

Kilian Q. Weinberger
Cornell University
kqw4@cornell.edu

Abstract

Recent work has shown that convolutional networks can be substantially deeper, more accurate, and efficient to train if they contain shorter connections between layers close to the input and those close to the output. In this paper, we embrace this observation and introduce the Dense Convolutional Network (DenseNet), which connects each layer to every other layer in a feed-forward fashion. Whereas traditional convolutional networks with L layers have L connections—one between each layer and its subsequent layer—our network has $\frac{L(L+1)}{2}$ direct connections. For each layer, the feature-maps of all preceding layers are used as inputs, and its own feature-maps are used as inputs into all subsequent layers. DenseNets have several compelling advantages: they alleviate the vanishing-gradient problem, strengthen feature propagation, encourage feature reuse, and substantially reduce the number of parameters. We evaluate our proposed architecture on four highly competitive object recognition benchmark tasks (CIFAR-10, CIFAR-100, SVHN, and ImageNet). DenseNets obtain significant improvements over the state-of-the-art on most of them, whilst requiring less computation to achieve high performance. Code and pre-trained models are available at <https://github.com/liuzhuang13/DenseNet>.

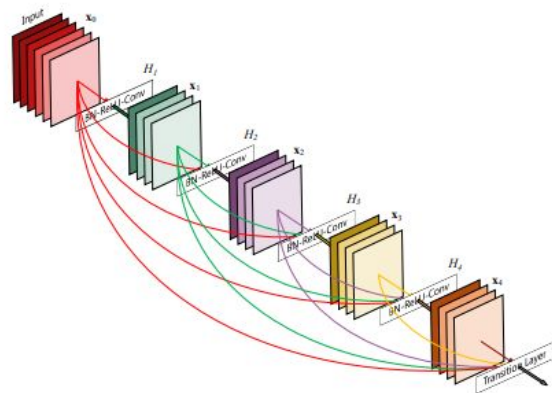
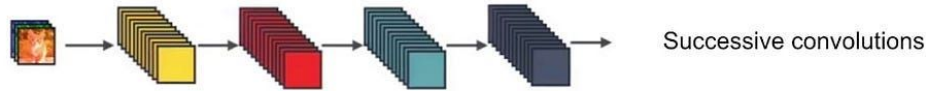


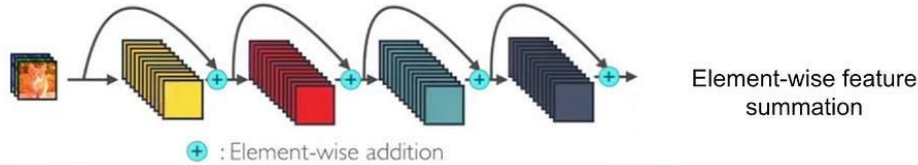
Figure 1: A 5-layer dense block with a growth rate of $k = 4$. Each layer takes all preceding feature-maps as input.

DenseNet

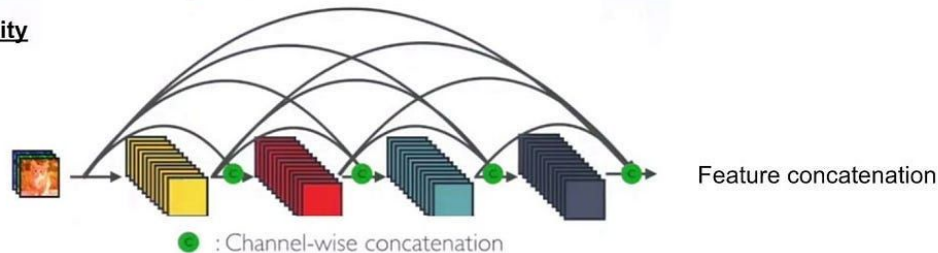
Standard Connectivity



Resnet Connectivity



DenseNet Connectivity



Estándar: Flujo lineal. No hay reutilización de características intermedias.

ResNet: Conexiones directas por suma (+). Conserva la información del gradiente al crear un "camino directo" para que fluya la señal, pero la suma puede alterar o mezclar la información explícita de los mapas de características previos.

DenseNet: Conexiones directas por concatenación. Conserva toda la información intacta en canales separados, permitiendo que cada capa combine explícitamente rasgos de diferentes niveles de abstracción.

ResNeXt

ResNeXt (2017) es una arquitectura de red neuronal altamente modular para la clasificación de imágenes que surge como una evolución de ResNet. Su innovación principal es la introducción de una nueva dimensión fundamental en el diseño de redes llamada "**cardinalidad**" (cardinality), que define el número de transformaciones paralelas (o ramas) que ocurren dentro de un mismo bloque.

Inspirada en la estrategia de "dividir, transformar y fusionar" (split-transform-merge) utilizada por los modelos Inception, ResNeXt simplifica este enfoque asegurando que todas estas ramas paralelas compartan exactamente la misma topología, lo que facilita su diseño y requiere elegir menos hiperparámetros

stage	output	ResNet-50	ResNeXt-50 (32×4d)
conv1	112×112	7×7, 64, stride 2	7×7, 64, stride 2
conv2	56×56	3×3 max pool, stride 2	3×3 max pool, stride 2
		$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128, C=32 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3	28×28	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256, C=32 \\ 1 \times 1, 512 \end{bmatrix} \times 4$
conv4	14×14	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512, C=32 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$
conv5	7×7	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 1024 \\ 3 \times 3, 1024, C=32 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	global average pool 1000-d fc, softmax	global average pool 1000-d fc, softmax
# params.		25.5×10^6	25.0×10^6
FLOPs		4.1×10^9	4.2×10^9

Table 1. (Left) ResNet-50. (Right) ResNeXt-50 with a 32×4d template (using the reformulation in Fig. 3(c)). Inside the brackets are the shape of a residual block, and outside the brackets is the number of stacked blocks on a stage. “C=32” suggests grouped convolutions [24] with 32 groups. The numbers of parameters and FLOPs are similar between these two models.

Aggregated Residual Transformations for Deep Neural Networks

Saining Xie¹Ross Girshick²Piotr Dollár²Zhuowen Tu¹Kaiming He²¹UC San Diego²Facebook AI Research

{s9xie, ztu}@ucsd.edu

{rbg, pdollar, kaiminghe}@fb.com

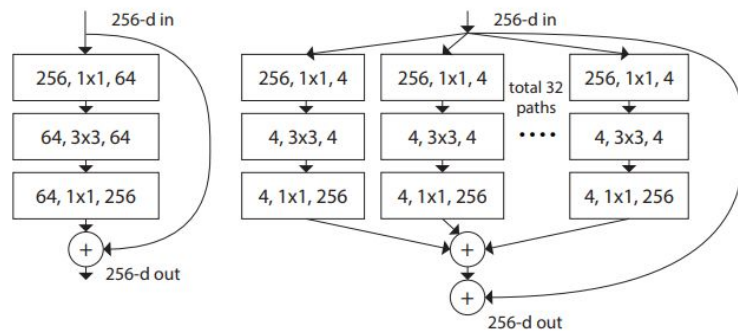


Figure 1. **Left:** A block of ResNet [14]. **Right:** A block of ResNeXt with cardinality = 32, with roughly the same complexity. A layer is shown as (# in channels, filter size, # out channels).

Abstract

We present a simple, highly modularized network architecture for image classification. Our network is constructed by repeating a building block that aggregates a set of transformations with the same topology. Our simple design results in a homogeneous, multi-branch architecture that has only a few hyper-parameters to set. This strategy exposes a new dimension, which we call “cardinality” (the size of the set of transformations), as an essential factor in addition to the dimensions of depth and width. On the ImageNet-1K dataset, we empirically show that even under the restricted condition of maintaining complexity, increasing cardinality is able to improve classification accuracy. Moreover, increasing cardinality is more effective than going deeper or wider when we increase the capacity. Our models, named ResNeXt, are the foundations of our entry to the ILSVRC 2016 classification task in which we secured 2nd place. We further investigate ResNeXt on an ImageNet-5K set and the COCO detection set, also showing better results than its ResNet counterpart. The code and models are publicly available online¹.

Xception

Xception es una arquitectura de red neuronal convolucional de alto rendimiento desarrollada por François Chollet. Su innovación principal y la base fundamental de su diseño es el uso de **convoluciones separables en profundidad** (depthwise separable convolutions) como reemplazo de las capas de convolución tradicionales.

La arquitectura Xception separa el proceso en dos etapas matemáticas: primero, realiza una convolución espacial de manera completamente independiente sobre cada canal de entrada; luego, emplea una **convolución puntual de 1x1** (pointwise convolution) para mezclar los canales resultantes. Este diseño parte de la hipótesis de que las ubicaciones espaciales de una imagen y sus canales de características están altamente desacoplados y pueden aprenderse de forma independiente.

Figure 1. A canonical Inception module (Inception V3).

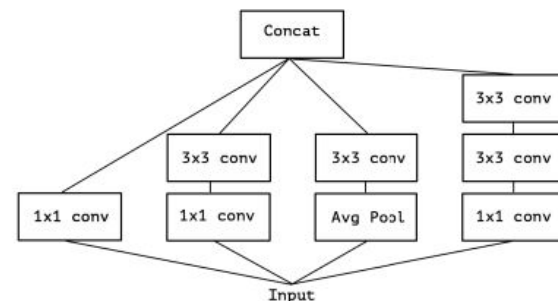
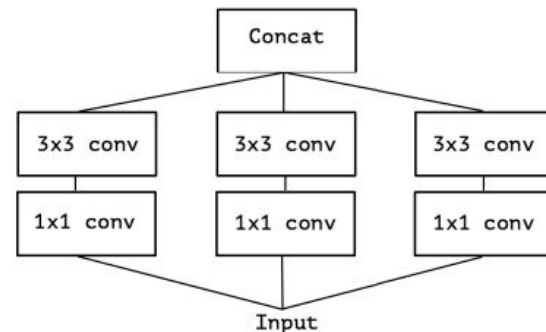


Figure 2. A simplified Inception module.



Xception: Deep Learning with Depthwise Separable Convolutions

François Chollet
Google, Inc.

fchollet@google.com

Abstract

We present an interpretation of Inception modules in convolutional neural networks as being an intermediate step in-between regular convolution and the depthwise separable convolution operation (a depthwise convolution followed by a pointwise convolution). In this light, a depthwise separable convolution can be understood as an Inception module with a maximally large number of towers. This observation leads us to propose a novel deep convolutional neural network architecture inspired by Inception, where Inception modules have been replaced with depthwise separable convolutions. We show that this architecture, dubbed Xception, slightly outperforms Inception V3 on the ImageNet dataset (which Inception V3 was designed for), and significantly outperforms Inception V3 on a larger image classification dataset comprising 350 million images and 17,000 classes. Since the Xception architecture has the same number of parameters as Inception V3, the performance gains are not due to increased capacity but rather to a more efficient use of model parameters.

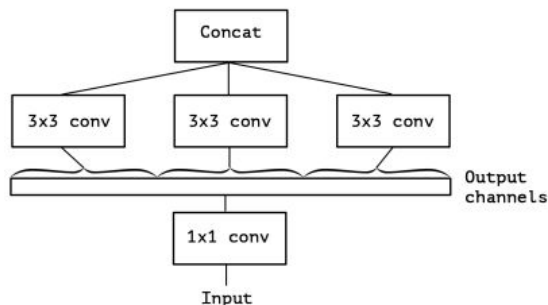


Figure 4. An “extreme” version of our Inception module, with one spatial convolution per output channel of the 1x1 convolution.

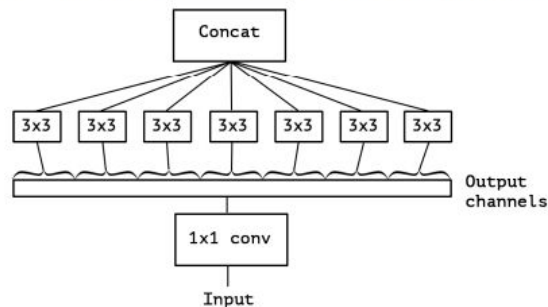
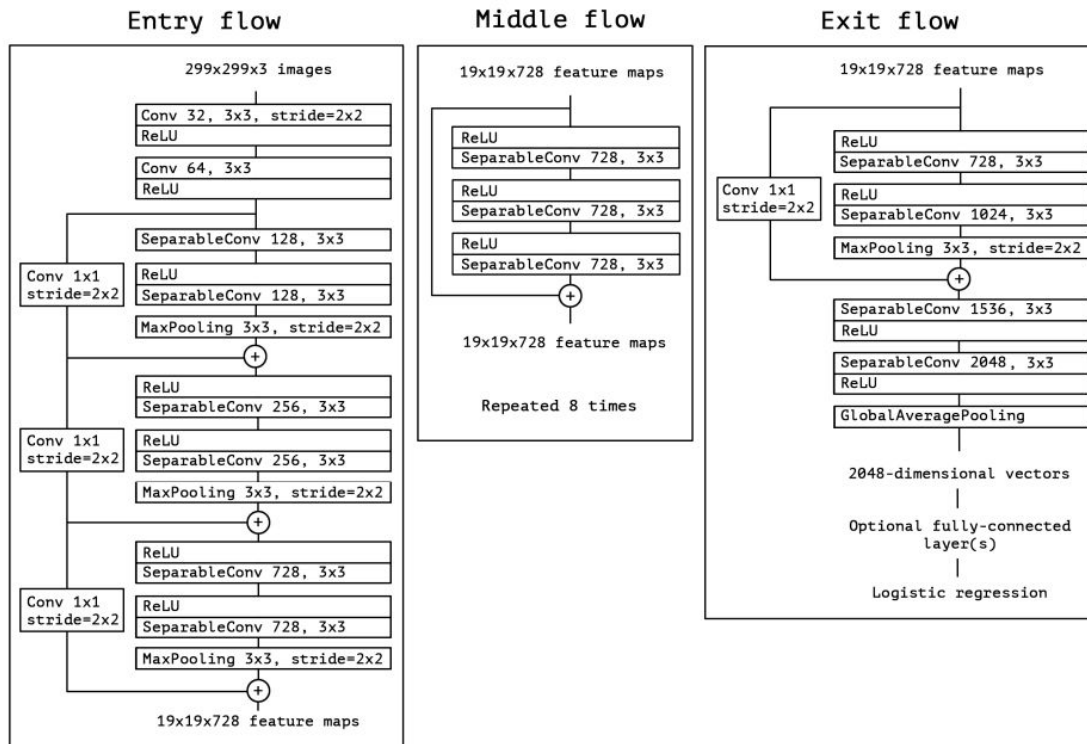
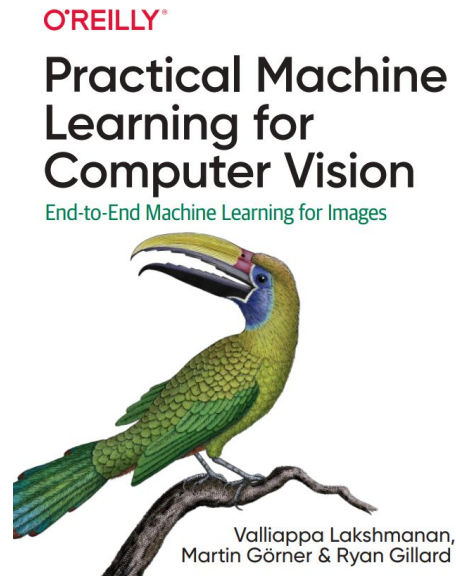
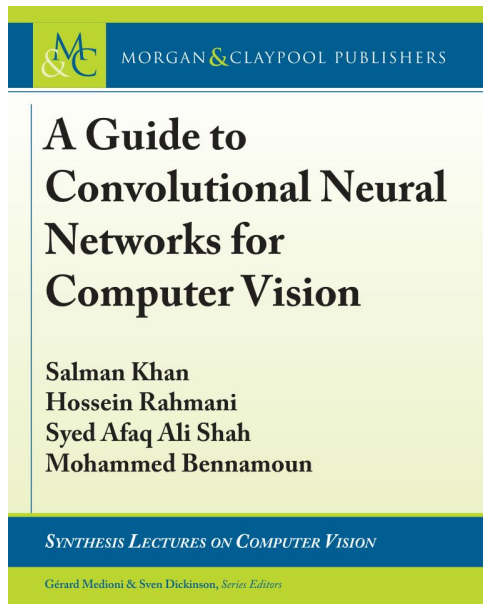
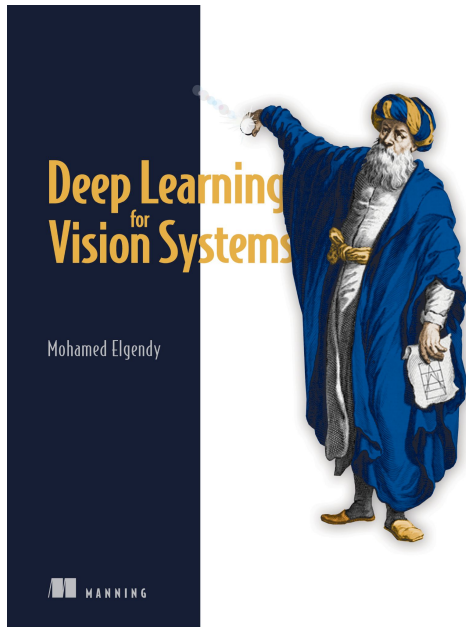


Figure 5. The Xception architecture: the data first goes through the entry flow, then through the middle flow which is repeated eight times, and finally through the exit flow. Note that all Convolution and SeparableConvolution layers are followed by batch normalization [7] (not included in the diagram). All SeparableConvolution layers use a depth multiplier of 1 (no depth expansion).

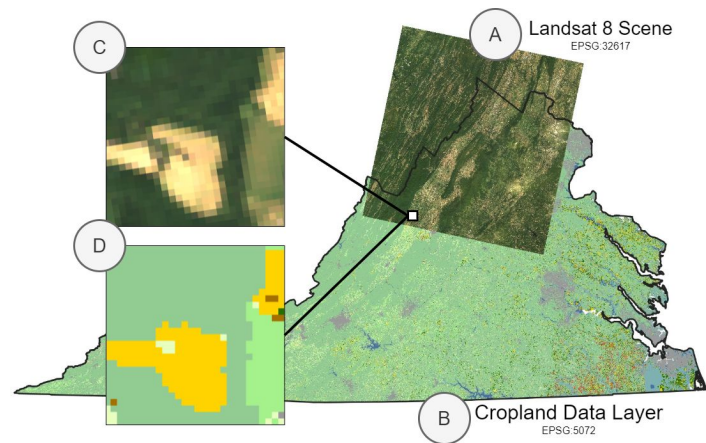
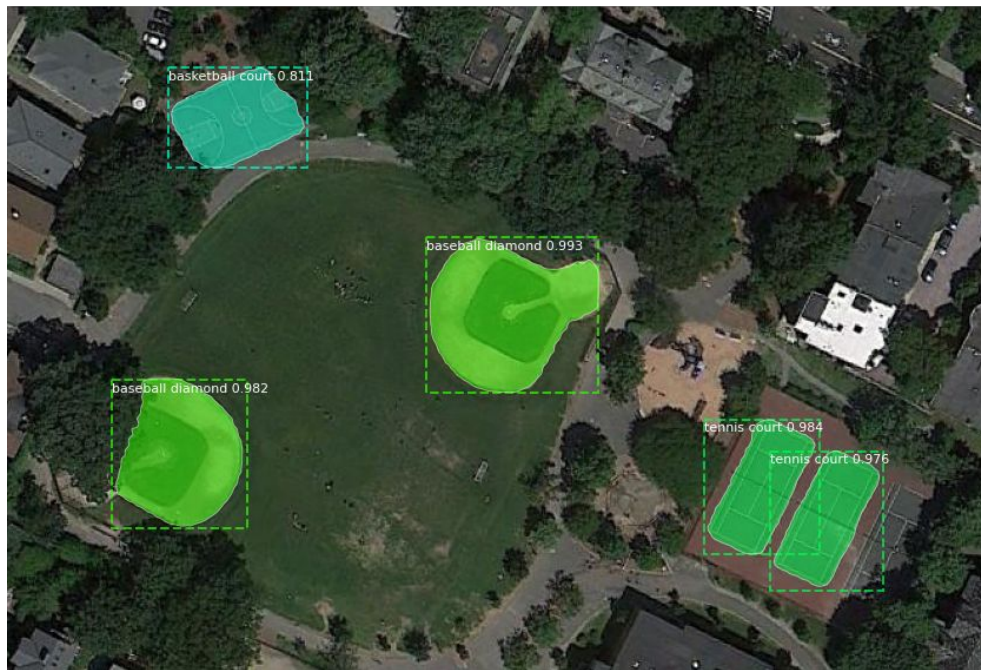


Para profundizar en estas arquitecturas



Aplicaciones Espaciales

Librería TorchGeo



<https://pytorch.org/blog/geospatial-deep-learning-with-torchgeo/>

AI enabled geospatial intelligence for the energy transition: A comparative CNN study on CCUS, geothermal siting, and NetZero strategies

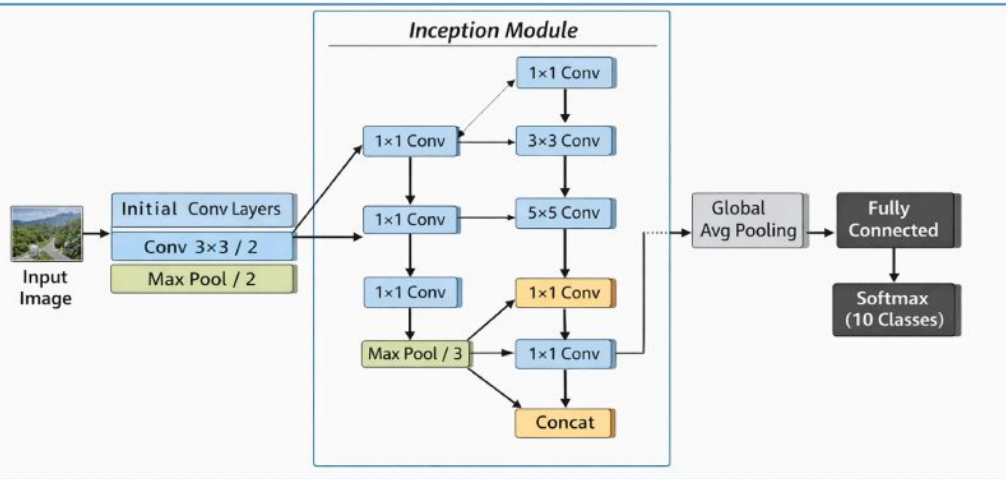


Fig. 8. Inception V3 architecture.

Table 3

Training Configuration and architectural complexity.

Model	Parameters (Millions)	Input Size	Training Epochs	Optimizer
VGG19	~143	$224 \times 224 \times 3$	30	Adaptive Moment Estimation;
ResNet	~25	$224 \times 224 \times 3$	30	Adaptive Moment Estimation;
Inception	~23	$224 \times 224 \times 3$	30	Adaptive Moment Estimation;
MobileNet	~4	$224 \times 224 \times 3$	30	Adaptive Moment Estimation;

- Deep learning for vision systems.
- A guide to convolutional neural networks for computer vision
- Practical Machine Learning for Computer Vision
- <https://www.ikomia.ai/blog/mastering-resnet-deep-learning-image-recognition>
- <https://www.pinecone.io/learn/series/image-search/cnn/>